

编译原理期末大作业实验报告

类 Rust 语言语义分析与中间代码生成器设计与实现

学院： 计算机科学与技术学院

专业： 计算机科学与技术

学号： 2351579 / 2253797 / 2352628

姓名： 程浩然 / 黄艺鑫 / 韦畅

指导教师： 高珍、卫志华

学期： 2026 年春季学期

2026 年 6 月 5 日

目录

摘要	4
1 概述	5
1.1 项目背景与目标	5
1.2 前一阶段工作基础	5
1.3 新增模块概述	7
1.3.1 语义分析器 (SemanticAnalyzer)	7
1.3.2 中间代码生成器 (IRGenerator)	7
1.4 团队分工与开发环境	7
2 语义分析器设计与实现	9
2.1 语义分析理论基础	9
2.1.1 语义分析的任务	9
2.1.2 Visitor 模式	9
2.2 类型系统设计	11
2.2.1 类型体系	11
2.2.2 类型定义与工厂方法	12
2.2.3 类型从语法树节点的解析	13
2.2.4 类型等价性判断	14
2.2.5 类型推导机制	15
2.3 符号表设计	15
2.3.1 符号表条目	15
2.3.2 栈式作用域	16
2.3.3 函数信息管理	17
2.4 两遍扫描处理函数前向引用	17
2.5 语义错误检测	19
2.5.1 错误检测总览	19
2.5.2 类型检查实现	19

目录	2
2.5.3 借用追踪实现	22
2.5.4 循环深度计数与控制流检查	23
2.5.5 复合类型的匹配检查	23
2.5.6 左值检查详解	24
2.6 语义分析流程总览	25
3 中间代码生成器设计与实现	28
3.1 中间代码理论基础	28
3.1.1 为什么需要中间代码	28
3.1.2 四元式 (Quadruple)	28
3.2 IR 操作码定义	30
3.3 四元式数据结构	30
3.4 IR 生成器核心实现	31
3.4.1 临时变量与标签管理	31
3.4.2 表达式 IR 生成	32
3.4.3 控制流语句的 IR 生成	33
3.4.4 函数调用的 IR 生成	37
3.4.5 引用与解引用的 IR 生成	38
3.5 完整的 IR 生成示例	39
3.6 IR 生成流程总览	41
4 测试与结果分析	42
4.1 测试策略	42
4.2 测试用例设计	42
4.2.1 综合正确用例: test_full.rs	42
4.2.2 语义错误检测用例: test_errors.rs	43
4.3 批量测试脚本与结果	44
4.4 运行效果	46
5 思考与分析	47

目录	3
5.1 语义分析中 Visitor 模式的设计考量	47
5.1.1 为什么选择 Visitor 模式	47
5.1.2 表达式类型传播的设计	47
5.2 两遍扫描 vs 单遍扫描的权衡	47
5.3 借用追踪的简化实现与局限	48
5.4 四元式 vs 其他 IR 形式的对比	49
5.5 控制流翻译中标签管理的设计	49
5.6 语义错误的组织与报告策略	49
5.7 类型系统的设计哲学：结构化 vs 命名化	50
5.8 IR 操作码设计中的冗余与完备性	50
6 总结与反思	52
6.1 项目成果总结	52
6.2 遇到的困难与解决方案	52
6.3 知识收获与能力提升	53
6.4 流水线编排与模块协作	53
6.5 改进方向与展望	54
6.6 个人想法与体会	55
6.6.1 对语义分析在编译器中地位的认识	55
6.6.2 对四元式设计的反思	55
6.6.3 对渐进式文法设计的理解	55
7 参考文献	57
附录 A 完整 IR 操作码定义表	58
附录 B 完整测试用例索引	59

摘要

本报告是编译原理课程期末大作业的实验报告，在前一阶段词法分析器与递归下降语法分析器的基础上，进一步实现了**语义分析器**和**中间代码生成器**。语义分析器基于 Visitor 模式遍历语法树，实现了符号表管理、类型检查、借用追踪等完整的语义分析功能，能够检测 18 类语义错误。中间代码生成器在语义分析通过后运行，将语法树转换为四元式 (Quadruple) 序列，覆盖算术运算、比较运算、控制流跳转、函数调用、数组操作、引用与解引用等语言特性。

本报告详细阐述了语义分析和中间代码生成的理论基础、系统设计、各模块的具体实现方案，并通过完整的测试用例（22 项测试全覆盖）验证了系统的正确性。

关键词：语义分析、中间代码生成、四元式、符号表、类型检查、Visitor 模式、编译原理、类 Rust 语言

1 概述

1.1 项目背景与目标

编译原理课程的大作业分两个阶段进行。在前一阶段（大作业 1），本小组已完成了类 Rust 语言的词法分析器（基于 Flex）和递归下降语法分析器，能够将源代码字符串转换为语法树。本阶段（大作业 2）的目标是在此基础上，实现编译器的后续两个核心阶段——语义分析和中间代码生成。

本阶段的核心目标是：

1. 设计并实现一个完整的**语义分析器**，能够在语法树基础上进行类型检查、作用域分析和借用追踪
2. 设计并实现一个**中间代码生成器**，能够将语法树转换为四元式形式的中间代码
3. 理解并应用语义分析（符号表、类型系统、Visitor 模式）和中间代码生成（三地址码、控制流翻译）的理论基础
4. 检测并报告 18 类静态语义错误，确保中间代码的正确性

1.2 前一阶段工作基础

大作业 1 已完成的模块为本阶段提供了坚实的基础：

表 1: 大作业 1 已完成的模块

模块	技术方案	输出
词法分析器	Flex + 正则表达式	TSV 格式 Token 流
语法分析器	递归下降（LL(1)）	缩进式语法树
TokenStream	流式接口（peek/advance）	词法到语法的桥梁
GUI 可视化	ImGui + GLFW + OpenGL3	代码/Token/语法树三栏展示

本阶段的工作直接复用大作业 1 的语法树结构，在语法树上运行语义分析 Visitor，然后运行中间代码生成 Visitor。

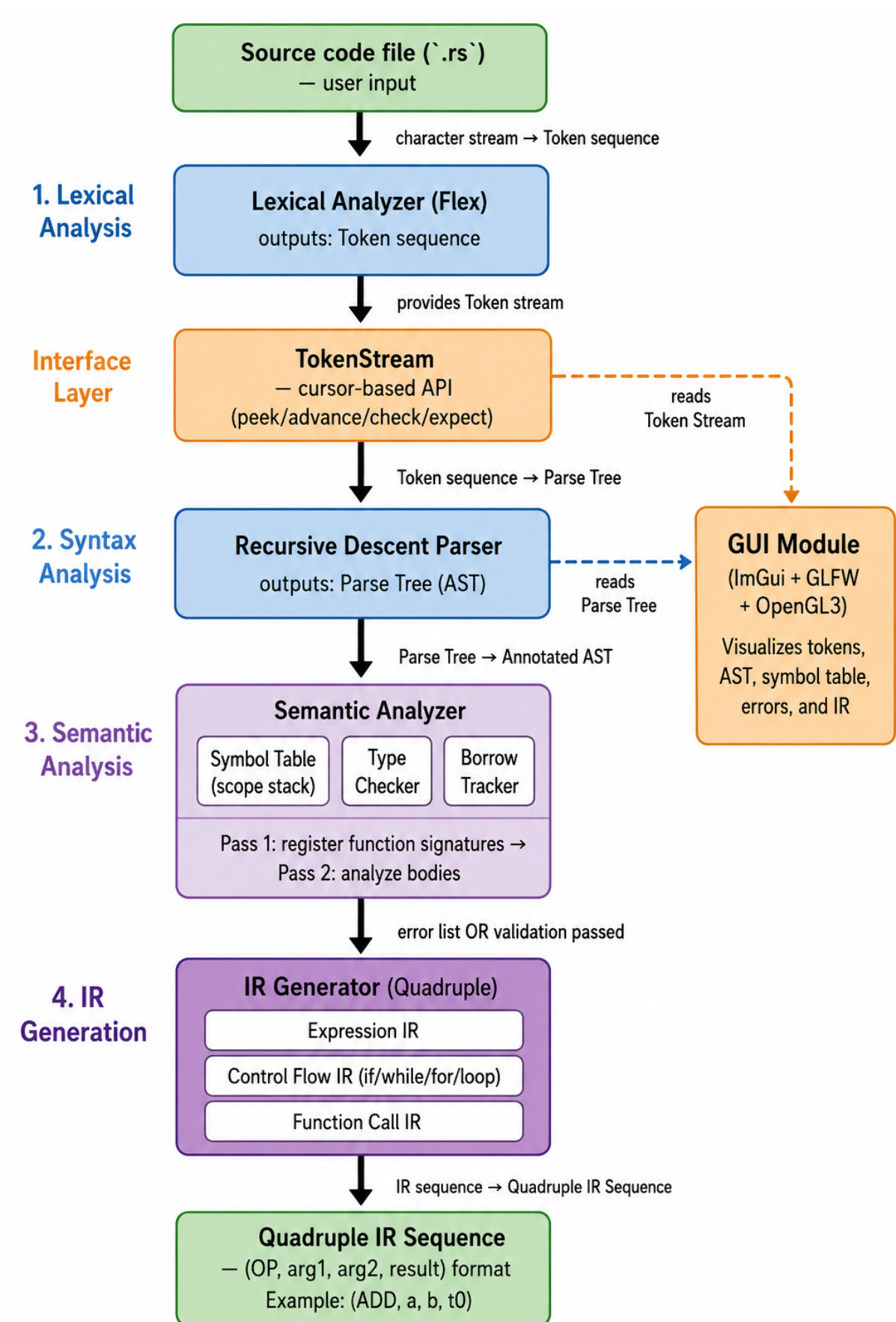


图 1: 系统总体架构图 (含语义分析与中间代码生成)

1.3 新增模块概述

1.3.1 语义分析器 (SemanticAnalyzer)

语义分析器继承了大作业 1 中讨论的”语义约束”检测思路，实现了完整的类型检查和语义错误诊断。主要功能包括：

- **符号表管理**：使用栈式作用域，支持变量声明、查询、shadowing
- **类型系统**：支持 i32、bool、void、引用 (&T / &mut T)、数组 ([T;N])、元组 ((T1,T2,...)) 共 7 种基础类型
- **类型检查**：表达式类型推导、赋值类型一致性、函数参数/返回值类型匹配
- **借用追踪**：不可变引用/可变引用的共存约束检查
- **错误诊断**：18 类语义错误的检测与定位

1.3.2 中间代码生成器 (IRGenerator)

中间代码生成器在语义分析通过后运行，将语法树转换为四元式序列。主要设计决策包括：

- **四元式形式**：(OP, arg1, arg2, result)，直观、易于理解和后续优化
- **临时变量管理**：自动生成 t0, t1, t2... 等临时变量持有中间计算结果
- **标签管理**：自动生成 L0, L1, L2... 等标签用于控制流跳转
- **27 种操作码**：覆盖赋值、算术、比较、跳转、函数调用、数组索引、引用/解引用等

1.4 团队分工与开发环境

表 2: 团队分工表

姓名	学号	主要分工
程浩然	2351579	语义分析器设计实现（类型系统、符号表、18 类错误检测）、借用追踪机制、两遍扫描函数前向引用处理、测试用例编写
黄艺鑫	2253797	中间代码生成器设计实现（四元式体系、控制流翻译）、表达式 IR 生成（运算符优先级映射）、IR 输出格式设计
韦畅	2352628	实验报告撰写、语义规则整理与对照、测试用例设计验证、文档与参考资料整理

表 3: 开发环境配置

组件	版本/说明
操作系统	Linux (Ubuntu 22.04) / Windows 11
编译器	GCC/G++ 11, C++17
Flex 版本	2.6.4
CMake 版本	3.22+
容器环境	Docker (Ubuntu 22.04 基础镜像)
GUI 框架	ImGui + GLFW + OpenGL3 (仅 Windows)

2 语义分析器设计与实现

语义分析是编译过程的第三阶段，负责在语法树的基础上检查程序的静态语义正确性。本章详细介绍语义分析器的设计与实现。

2.1 语义分析理论基础

2.1.1 语义分析的任务

语义分析位于语法分析和中间代码生成之间，其核心任务是：

1. **类型检查**：验证每个表达式、语句、声明的类型是否一致
2. **作用域分析**：确定每个标识符的可见范围，处理变量声明与引用
3. **控制流检查**：验证 break/continue 在循环内、return 类型匹配等
4. **借用规则检查**：验证 Rust 风格的所有权和借用规则

2.1.2 Visitor 模式

本项目的语义分析采用 **Visitor 模式**（访问者模式）实现。Visitor 模式的核心思想是将操作（语义分析）与数据结构（语法树）分离：语法树节点提供 accept 方法接收 Visitor，Visitor 为每种节点类型提供对应的 visit 方法。

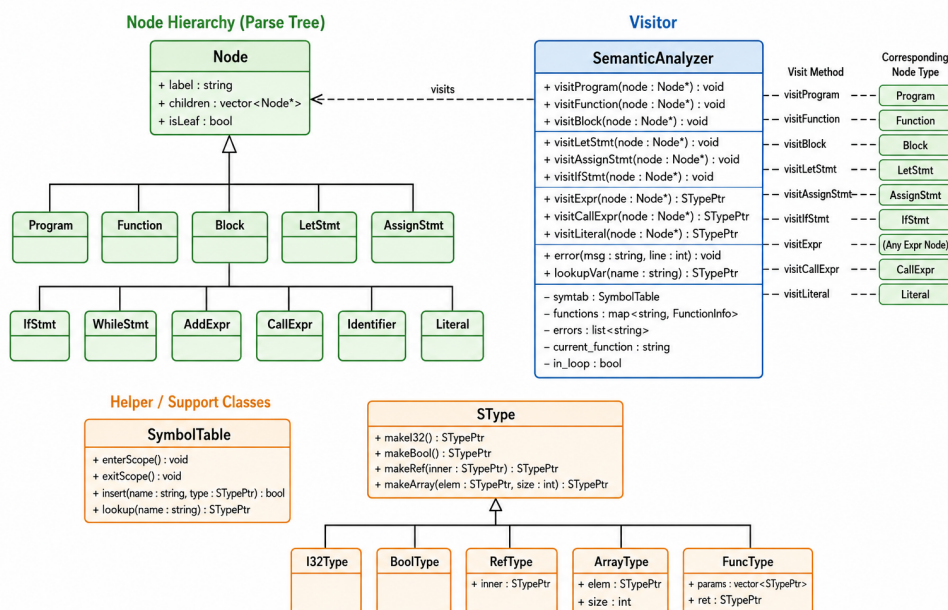


图 2: Visitor 模式在语义分析中的应用

在我们的实现中，Visitor 模式体现为一系列 visit 函数：

```
1 // 顶层访问
2 void visitProgram(const Node* node);
3 void visitFunction(const Node* node);
4 void visitBlock(const Node* node);
5 void visitStmt(const Node* node);
6
7 // 语句访问
8 void visitLetStmt(const Node* node);
9 void visitAssignStmt(const Node* node);
10 void visitReturnStmt(const Node* node);
11 void visitIfStmt(const Node* node);
12 // ...
13
14 // 表达式访问（返回类型信息）
15 STypePtr visitExpr(const Node* node);
16 STypePtr visitCmpExpr(const Node* node);
17 STypePtr visitCallExpr(const Node* node);
18 // ...
```

Listing 1: Visitor 模式的函数声明

表达式 Visitor 返回类型信息 (STypePtr)，沿语法树自底向上传播，实现类型推导和类型检查。

Visitor 模式的工作机制可以通过一个具体例子来理解。考虑语句 `let mut x = a + 1;` 的语义分析过程：

1. `visitStmt()` 遇到 `LetStmt` 节点 → 分发到 `visitLetStmt()`
2. `visitLetStmt()` 提取变量名“x”，检测到无类型注解（类型初始为 UNKNOWN），检测到有初始化表达式
3. 对初始化表达式子树调用 `visitExpr()`:
 - 遇到 `AddExpr` 节点 → 分发到 `visitAddExpr()`
 - `visitAddExpr()` 对左操作数调用 `visitExpr()`:
 - 遇到 `Identifier("a")` → `lookupVar("a")` → 返回 `Symbol(类型 =ST_I32, is_assigned=true)`
 - `visitAtom()` 返回 `ST_I32`

- 对右操作数调用 `visitExpr()`:
 - 遇到 `Literal(1)` $\rightarrow$ `visitLiteral()` $\rightarrow$ 返回 `ST_I32`
- 检查左操作数类型 (`ST_I32`) 与右操作数类型 (`ST_I32`) 是否一致 $\rightarrow$ 一致
- `visitAddExpr()` 返回 `ST_I32` (算术运算的结果类型)

4. `visitLetStmt()` 收到 `initType = ST_I32`

5. 变量 `x` 的类型从 `UNKNOWN` 更新为 `ST_I32` (类型推导完成)

6. 创建 `Symbol("x", ST_I32, mut=true, is_assigned=true)` 并插入符号表

这个例子展示了 Visitor 模式的核心优势：类型信息沿着语法树结构自底向上自然流动，每个 `visit` 函数只需关注自己这一层的处理逻辑，下层细节被递归调用封装。

2.2 类型系统设计

2.2.1 类型体系

类型系统是语义分析的核心。我们设计了完整的基础类型层次结构：

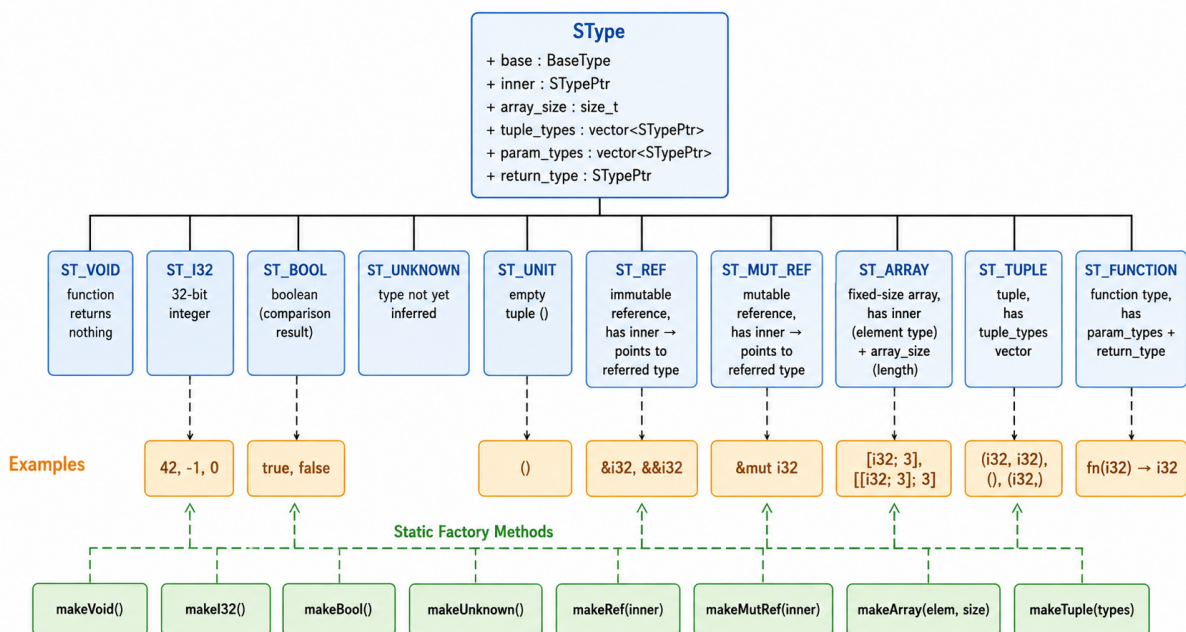


图 3: 语义类型系统层次结构

表 4: 基础类型 (BaseType) 枚举

BaseType	说明	示例
ST_VOID	空类型 (无返回值)	函数返回 void
ST_I32	32 位整数	i32
ST_BOOL	布尔类型 (比较运算结果)	1==1 的类型
ST_REF	不可变引用	&i32
ST_MUT_REF	可变引用	&mut i32
ST_ARRAY	定长数组	[i32; 3]
ST_TUPLE	元组	(i32, i32)
ST_UNIT	单元类型 (空元组)	()
ST_FUNCTION	函数类型	fn(i32) -> i32
ST_UNKNOWN	未知类型 (类型推导暂未完成)	声明时未指定类型

2.2.2 类型定义与工厂方法

```

1 class SType {
2 public:
3     BaseType base = BaseType::ST_UNKNOWN; // 基础类型
4     STypePtr inner;                       // 引用/数组的内部类型
5     int array_size = 0;                   // 数组大小
6     vector<STypePtr> tuple_types;         // 元组元素类型
7     vector<STypePtr> param_types;         // 函数参数类型
8     STypePtr return_type;                 // 函数返回类型
9
10    // 工厂方法
11    static STypePtr makeVoid();
12    static STypePtr makeI32();
13    static STypePtr makeBool();
14    static STypePtr makeRef(STypePtr inner);
15    static STypePtr makeMutRef(STypePtr inner);
16    static STypePtr makeArray(STypePtr elem, int size);
17    static STypePtr makeTuple(vector<STypePtr> types);
18    static STypePtr makeUnknown();
19
20    // 类型判断
21    bool isI32() const;
22    bool isBool() const;
23    bool isRef() const;

```

```

24     bool isArray() const;
25     // ...
26
27     // 类型比较
28     bool equals(const STypePtr& other) const;
29     string toString() const;
30 };

```

Listing 2: SType 类定义

工厂方法模式使类型创建语义清晰，避免手动设置内部状态可能导致的错误。

2.2.3 类型从语法树节点的解析

类型信息出现在源代码的多个位置：变量声明、函数参数、返回类型等。所有这些位置都通过 `parseTypeFromNode()` 方法统一处理：

```

1 STypePtr parseTypeFromNode(const Node* node) {
2     // 需要 Type 标签
3     if (!node || node->label != "Type") return SType::makeUnknown();
4
5     auto& first = node->children[0];
6
7     // i32 -> ST_I32
8     if (isLeafCat(first, "I32")) return SType::makeI32();
9
10    // &T -> ST_REF, &mut T -> ST_MUT_REF
11    if (isLeafCat(first, "Ampersand")) { /* ... */ }
12
13    // [T; N] -> ST_ARRAY
14    if (isLeafCat(first, "LBracket")) { /* ... */ }
15
16    // (T1, T2, ...) -> ST_TUPLE
17    if (isLeafCat(first, "LParen")) { /* ... */ }
18
19    return SType::makeUnknown();
20 }

```

Listing 3: 类型解析逻辑

该方法递归解析嵌套类型，例如 `[[i32;3];3]` 会被解析为 `ST_ARRAY(inner=ST_ARRAY(inner=ST_I32, size=3), size=3)`。

2.2.4 类型等价性判断

类型检查的核心操作是判断两个类型是否兼容。我们实现了 `equals()` 方法来进行结构化的类型比较：

```

1 bool SType::equals(const STypePtr& other) const {
2     if (!other) return false;
3     if (base != other->base) return false;
4
5     switch (base) {
6     case BaseType::ST_REF:
7     case BaseType::ST_MUT_REF:
8         // 引用类型：要求内部类型相同，且引用类别相同
9         return inner && other->inner && inner->equals(other->inner);
10
11    case BaseType::ST_ARRAY:
12        // 数组类型：要求元素类型相同 且 长度相同
13        return array_size == other->array_size
14            && inner && other->inner && inner->equals(other->inner);
15
16    case BaseType::ST_TUPLE:
17        // 元组类型：要求每个位置的元素类型一一对应相同
18        if (tuple_types.size() != other->tuple_types.size())
19            return false;
20        for (size_t i = 0; i < tuple_types.size(); i++)
21            if (!tuple_types[i]->equals(other->tuple_types[i]))
22                return false;
23        return true;
24
25    default:
26        // 基础类型 (VOID/I32/BOOL/UNIT/UNKNOWN)：直接比较枚举值
27        return true;
28    }
29 }
```

Listing 4: 类型等价性判断

类型等价判断的设计要点：

- **结构等价 (Structural Equivalence)**：不是比较类型名称（名字等价），而是递归比较类型的内部结构。例如两个 `[i32; 3]` 即使来源于不同的类型声明，也被判定

为同一类型

- **引用类别敏感**: 不可变引用 `&T` 和可变引用 `&mut T` 是不等价的, 即使内部类型 `T` 相同
- **ST_UNKNOWN 宽松匹配**: 当某一侧的类型尚为 UNKNOWN (如未初始化的类型推导), `equals()` 在该侧成立时返回 `true`, 避免在类型推导完成前误报错误

2.2.5 类型推导机制

当变量声明未指定类型时 (如 `let mut b;`), 语义分析器需要通过后续的赋值来推导类型。我们采用以下策略:

1. **声明时标记为 UNKNOWN**: 变量初次声明时类型设为 `ST_UNKNOWN`
2. **首次赋值触发推导**: 当对 UNKNOWN 类型变量赋值时, 变量类型更新为赋值表达式的类型
3. **后续赋值做类型检查**: 类型确定后, 后续赋值需进行常规的类型一致性检查
4. **使用前检查**: 如果变量在类型推导完成前就被作为右值使用 (`is_assigned == false`), 报告”变量使用前未赋值”错误

这种设计模拟了 Rust 编译器的类型推导行为: 局部变量的类型可以根据首次使用的上下文自动推导, 不需要显式标注。

2.3 符号表设计

2.3.1 符号表条目

每个声明的变量在符号表中对应一个 Symbol 条目:

```
1 struct Symbol {
2     string name;           // 变量名
3     STyepPtr type;        // 变量类型
4     bool is_mutable;      // 是否可变 (mut)
5     bool is_assigned;     // 是否已被赋值
6     int scope_level;      // 声明所在作用域层级
7     int line;             // 声明行号
8     bool has_immutable_ref; // 是否存在不可变引用 (借用追踪)
9     bool has_mutable_ref;  // 是否存在可变引用 (借用追踪)
```

```
10 };
```

Listing 5: Symbol 结构体

2.3.2 栈式作用域

符号表采用栈式作用域（Stack of HashMaps）实现：

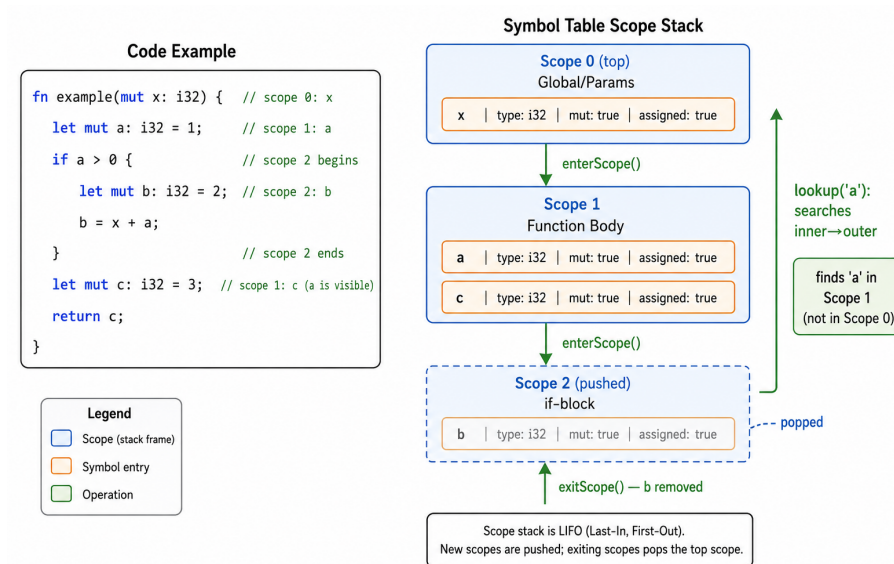


图 4: 符号表栈式作用域结构

```
1 class SymbolTable {
2     vector<unordered_map<string, shared_ptr<Symbol>>> scopes;
3     int current_scope = 0;
4
5     void enterScope() {
6         current_scope++;
7         scopes.emplace_back(); // 压入新作用域
8     }
9
10    void exitScope() {
11        scopes.pop_back(); // 弹出当前作用域
12        current_scope--;
13    }
14
15    shared_ptr<Symbol> lookup(const string& name) const {
16        // 从内向外逐层查找
17        for (int i = scopes.size() - 1; i >= 0; i--) {
```

```
18         auto it = scopes[i].find(name);
19         if (it != scopes[i].end()) return it->second;
20     }
21     return nullptr; // 未找到
22 }
23 };
```

Listing 6: SymbolTable 实现

栈式作用域的设计特点：

- 作用域嵌套：每个 {...} 块创建新的作用域层
- 从内向外查找：lookup() 从最内层开始向外搜索，自然实现变量遮蔽
- Shadowing 支持：insert() 直接覆盖当前作用域中的同名变量，实现 Rust 风格的重影（shadowing）
- 退出即清理：离开作用域时弹出栈顶，自动释放该作用域内的所有变量

2.3.3 函数信息管理

函数信息使用独立的 HashMap 管理（不随作用域消除），存储函数签名供调用点检查：

```
1 struct FunctionInfo {
2     string name; // 函数名
3     vector<pair<string, STypePtr>> params; // 形参列表 (name,
        type)
4     STypePtr return_type; // 返回类型
5     int line; // 声明行号
6 };
```

Listing 7: FunctionInfo 结构体

2.4 两遍扫描处理函数前向引用

本项目的一个重要设计是两遍扫描（Two-Pass）策略，允许函数在定义之前被调用：

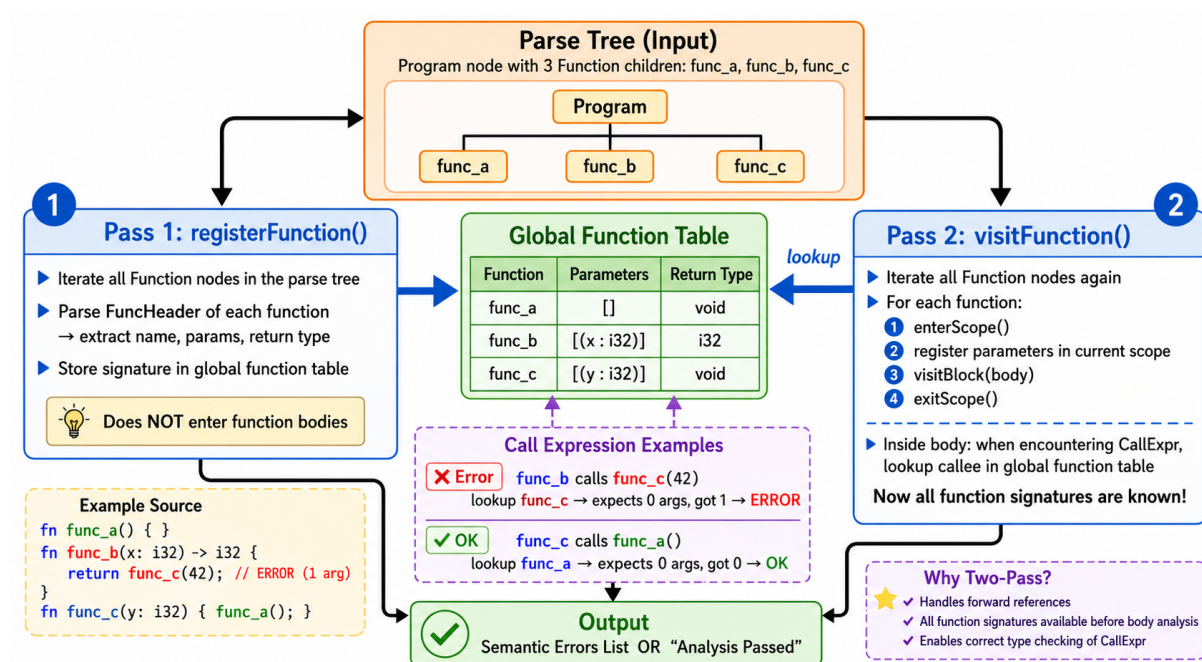


图 5: 两遍扫描策略处理函数前向引用

```
1 void visitProgram(const Node* node) {
2     auto funcNodes = findChildren(node, "Function");
3
4     // Pass 1: 注册所有函数签名（不分析函数体）
5     for (auto* fn : funcNodes)
6         registerFunction(fn);
7
8     // Pass 2: 分析所有函数体（此时所有函数签名已知）
9     for (auto* fn : funcNodes)
10        visitFunction(fn);
11 }
```

Listing 8: 两遍扫描实现

第一遍调用 `registerFunction()`，解析每个函数的头声明（函数名、形参类型、返回类型），将签名存入全局函数表。此时不进入函数体，不分析任何表达式。

第二遍调用 `visitFunction()`，此时所有函数签名都已注册，函数体中的任何调用都能在全局表中找到目标函数的签名，从而正确检查参数数量和类型。

这种设计消除了 C 语言中需要“前向声明”的限制，使编译器更接近现代语言的行为。

2.5 语义错误检测

2.5.1 错误检测总览

本项目的语义分析器能够检测 **18 类语义错误**，覆盖作业要求的所有场景：

表 5: 语义错误检测列表

编号	错误类型	对应规则	示例
1	返回类型不一致 (void→i32)	1.5	fn f()->i32 { return;}
2	返回类型不一致 (i32→void)	1.5	fn f() { return 1;}
3	赋值给未声明变量	2.2	a=32; (a 未声明)
4	赋值类型不一致	2.2	let mut a:i32; a=1==1;
5	右值未提前声明	2.2	let mut b:i32=a; (a 未声明)
6	右值未提前赋值	2.2	let mut a:i32; let b:i32=a;
7	不可变变量二次赋值	6.1	let c:i32=1; c=2;
8	实参数量不一致	3.5	f(1) (f 无参数)
9	无返回值函数作为右值	3.5	let mut a=void_func();
10	实参类型不一致	3.5	f(1==1) (f 期望 i32)
11	break 不在循环体内	5.4	函数中直接 break;
12	continue 不在循环体内	5.4	函数中直接 continue;
13	可变引用与其他引用共存	6.3	&a 和 &mut a 同时存在
14	从不可变变量创建可变引用	6.3	let a:i32=1; let mut b=&mut a;
15	对非引用类型解引用	6.4	let mut b=*a; (a 非 ref)
16	不可变引用修改指向数据	6.4	let b=&a; *b=2;
17	数组初始化元素数量不一致	8.2	a=[1,2,3]; (a 声明为 [i32;2])
18	元组初始化元素数量不一致	9.2	a=(1,2,3); (a 声明为 (i32,i32))

2.5.2 类型检查实现

类型检查的核心是**类型一致性判断**。在赋值、参数传递、返回语句等位置，比较期望类型与实际类型。下面以赋值语句为例展示完整的类型检查流程：

```

1 void visitAssignStmt(const Node* node) {
2     int line = extractLine(node);
3     // 步骤1: 找到赋值号的位置
4     int assignIdx = -1;
5     for (size_t i = 0; i < node->children.size(); i++) {
6         if (isLeafCat(node->children[i].get(), "Assign")) {
7             assignIdx = (int)i; break;

```



```

8         }
9     }
10    if (assignIdx < 0) return;
11
12    auto* lhsNode = node->children[0].get();
13
14    // 步骤2: 检查左值声明状态
15    // 对于标识符左值: a = expr 或 *p = expr
16    // 对于数组下标: arr[0] = expr
17    checkLvalue(lhsNode, line);
18
19    // 步骤3: 左值类型推导 + 可变性检查
20    STypePtr lhsType;
21    if (lhsNode->label == "Identifier") {
22        string name = extractId(lhsNode);
23        auto sym = symtab.lookup(name);
24        if (sym) {
25            lhsType = sym->type;
26            // 检查可变性
27            if (!sym->is_mutable)
28                error("cannot assign to immutable variable", line);
29            // 检查不可变引用赋值
30            if (sym->type->isRef() && !sym->type->isMutRef())
31                error("cannot assign through immutable reference",
32                    line);
33        }
34    } else {
35        lhsType = visitExpr(lhsNode); // DerefExpr / IndexExpr
36    }
37
38    // 步骤4: 右值类型推导
39    auto rhsType = visitExpr(node->children[assignIdx + 1].get());
40
41    // 步骤5: 类型一致性检查
42    if (lhsType && !lhsType->isUnknown()
43        && !rhsType->isUnknown()
44        && !lhsType->equals(rhsType)) {
45        error("type mismatch in assignment: " +
46            lhsType->toString() + " vs " + rhsType->toString(),
47            line);

```

```

46     }
47
48     // 步骤6: 标记变量为已赋值
49     if (lhsNode->label == "Identifier") {
50         string name = extractId(lhsNode);
51         auto sym = symtab.lookup(name);
52         if (sym) sym->is_assigned = true;
53     }
54 }

```

Listing 9: 赋值语句的完整类型检查实现

类型传播机制：表达式类型沿语法树自底向上传播——字面量返回对应的类型（整数常量 $\rightarrow$ ST_I32），标识符从符号表查找类型（未声明 $\rightarrow$ 报错，未赋值 $\rightarrow$ 报错），比较运算始终返回 ST_BOOL，算术运算要求操作数类型一致并返回操作数类型，函数调用返回函数声明的返回类型。

下面以三个典型场景为例，展示类型检查的完整数据流：

场景一：基本赋值 `a = 42`（`a` 为 `let mut a:i32`）

1. `checkLvalue("a")` $\rightarrow$ 查符号表，`a` 存在且声明为 `mut` $\rightarrow$ 通过
2. 求 `lhsType` $\rightarrow$ `lookup("a")` $\rightarrow$ ST_I32
3. 求 `rhsType` $\rightarrow$ `visitExpr(42)` $\rightarrow$ `visitLiteral` $\rightarrow$ ST_I32
4. `equals(ST_I32, ST_I32)` $\rightarrow$ `true` $\rightarrow$ 通过类型检查
5. 标记 `a.is_assigned = true`

场景二：类型不匹配 `a = 1==1`（`a` 为 `let mut a:i32`）

1. `checkLvalue("a")` $\rightarrow$ 通过
2. `lhsType` = ST_I32
3. `visitExpr(1==1)` $\rightarrow$ `visitCmpExpr` $\rightarrow$ 左右操作数均为 ST_I32 $\rightarrow$ 返回 ST_BOOL
4. `rhType` = ST_BOOL
5. `equals(ST_I32, ST_BOOL)` $\rightarrow$ `false` $\rightarrow$ 报错“type mismatch: i32 vs bool”

场景三：类型推导 `let mut b; b = 42`

1. visitLetStmt $\rightarrow$ b 的类型初始为 ST_UNKNOWN (未指定类型注解)
2. visitAssignStmt $\rightarrow$ lhsType = ST_UNKNOWN
3. rhsType = ST_I32
4. equals 条件: lhsType 是 UNKNOWN $\rightarrow$ 跳过类型检查 (不报错)
5. 同时 b.is_assigned 更新为 true, b.type 从 ST_UNKNOWN 更新为 ST_I32

2.5.3 借用追踪实现

借用追踪模拟了 Rust 的所有权规则, 通过符号表中的 has_immutable_ref 和 has_mutable_ref 标志位实现:

```

1 STypePtr visitRefExpr(const Node* node) {
2     // ...
3     if (is_mut) {
4         // 创建 &mut: 检查是否有其他引用存在
5         if (sym->has_immutable_ref || sym->has_mutable_ref)
6             error("cannot create mutable reference: other refs
7                 exist");
8         sym->has_mutable_ref = true;
9     } else {
10        // 创建 &: 检查是否有可变引用存在
11        if (sym->has_mutable_ref)
12            error("cannot create immutable ref: mutable ref exists");
13        sym->has_immutable_ref = true;
14    }
15    // ...
16 }
```

Listing 10: 借用追踪逻辑

借用规则总结:

- 多个不可变引用允许多个 &T 同时存在
- 可变引用独占存在 &mut T 时, 不能有任何其他引用
- 只能从可变变量创建可变引用 let a 不能创建 &mut a
- 不可变引用不能修改数据 *b=2 在 b 为 &T 时报错

2.5.4 循环深度计数与控制流检查

break 和 continue 必须出现在循环体内，这一检查通过一个简单的计数器实现：

```

1 int in_loop = 0; // 当前所在的循环嵌套深度
2
3 // 进入循环时递增
4 void visitWhileStmt(const Node* node) {
5     // ... 检查条件表达式 ...
6     in_loop++;
7     visitBlock(bodyBlock);
8     in_loop--;
9 }
10
11 // break/continue时检查
12 void visitBreakStmt(const Node* node) {
13     if (in_loop <= 0)
14         error("break statement must be inside a loop", line);
15 }
16 void visitContinueStmt(const Node* node) {
17     if (in_loop <= 0)
18         error("continue statement must be inside a loop", line);
19 }

```

Listing 11: in_loop 计数器管理

in_loop 本质上是一个语义分析专用的“上下文标志”，与 IR 生成阶段的 break_label/continue_label 形成了清晰的职责分离：语义分析负责检查“是否在循环内”的合法性，IR 生成负责实现“跳转到正确目标”的语义。

2.5.5 复合类型的匹配检查

数组和元组类型的语义检查涉及两个维度：**类型维度**（元素类型是否匹配）和**数量维度**（元素个数是否一致）。

数组初始化检查—以 let mut a:[i32;2]; a=[1,2,3]; 为例：

- 左值类型：通过 parseTypeFromNode 解析得到 ST_ARRAY(inner=ST_I32, size=2)
- 右值类型：visitArrayLit 遍历三个元素，每个返回 ST_I32，构造类型为 ST_ARRAY(inner=ST_I32, size=3)
- equals(ST_ARRAY(i32,2), ST_ARRAY(i32,3)) → array_size 2 3 → false → 报错

元组初始化检查—以 `let mut a:(i32,i32); a=(1,2,3);` 为例:

- 左值类型: `ST_TUPLE` with types `[ST_I32, ST_I32]`
- 右值类型: `visitTupleLit` 遍历三个元素后, 构造 `ST_TUPLE` with types `[ST_I32, ST_I32, ST_I32]`
- `equals` 比较时, 先比较 `tuple_types.size()` (2 3), 立即返回 `false` → 报错

这种结构化比较自然地处理了嵌套复合类型, 例如 `[[i32;2];3]` 与 `[[i32;3];2]` 虽然总元素数相同 (6 个), 但由于数组长度不同 (2 3), 类型比较会返回 `false`。

2.5.6 左值检查详解

左值 (Lvalue) 是可以出现在赋值号左侧的表达式。我们的语言中, 合法的左值包括:

- 标识符: `a` (前提: 已声明且可变)
- 数组下标: `arr[0]` (前提: 数组变量可变)
- 解引用: `*ptr` (前提: `ptr` 为可变引用)
- 元组字段: `tup.0` (前提: 元组变量可变)

`checkLvalue()` 方法递归检查左值的有效性:

```

1 void SemanticAnalyzer::checkLvalue(const Node* node, int line) {
2     if (node->label == "Identifier") {
3         // 简单变量: 检查是否已声明
4         string name = extractId(node);
5         auto sym = symtab.lookup(name);
6         if (!sym) error("variable '" + name +
7                         "' not declared", line);
8         return;
9     }
10
11     if (node->label == "IndexExpr") {
12         // 数组下标 arr[i]: 检查数组是否可变
13         string name = extractId(node);
14         auto sym = symtab.lookup(name);
15         if (sym && !sym->is_mutable)

```

```

16         error("cannot assign to element of "+
17               "immutable array", line);
18     return;
19 }
20
21 if (node->label == "DerefExpr") {
22     // 解引用 *ptr: 检查 ptr 是否为可变引用
23     const Node* inner = nullptr;
24     for (auto& c : node->children)
25         if (!c->isLeaf) { inner = c.get(); break; }
26
27     if (inner) {
28         auto innerType = visitExpr(inner);
29         if (innerType->isRef() && !innerType->isMutRef())
30             error("cannot assign through "+
31                   "immutable reference", line);
32     }
33     return;
34 }
35 }

```

Listing 12: 左值检查实现

左值检查是语义分析的第一道防线——在检查类型匹配之前，先确保赋值的目标本身是合法的。

2.6 语义分析流程总览

语义分析的完整流程：

1. 语法树根节点 `Program` $\rightarrow$ `visitProgram()`
2. 第一遍 (Pass 1)：遍历所有 `Function` 节点，调用 `registerFunction()` 注册函数签名到全局函数表。此过程只解析函数头声明（函数名、形参类型、返回类型），不进入函数体
3. 第二遍 (Pass 2)：遍历所有 `Function` 节点，调用 `visitFunction()` 分析函数体。每个函数体内部：
 - `enterScope()`：创建新的符号表作用域
 - 注册形参：将每个参数作为 `Symbol` 插入符号表（预标记 `is_assigned = true`）

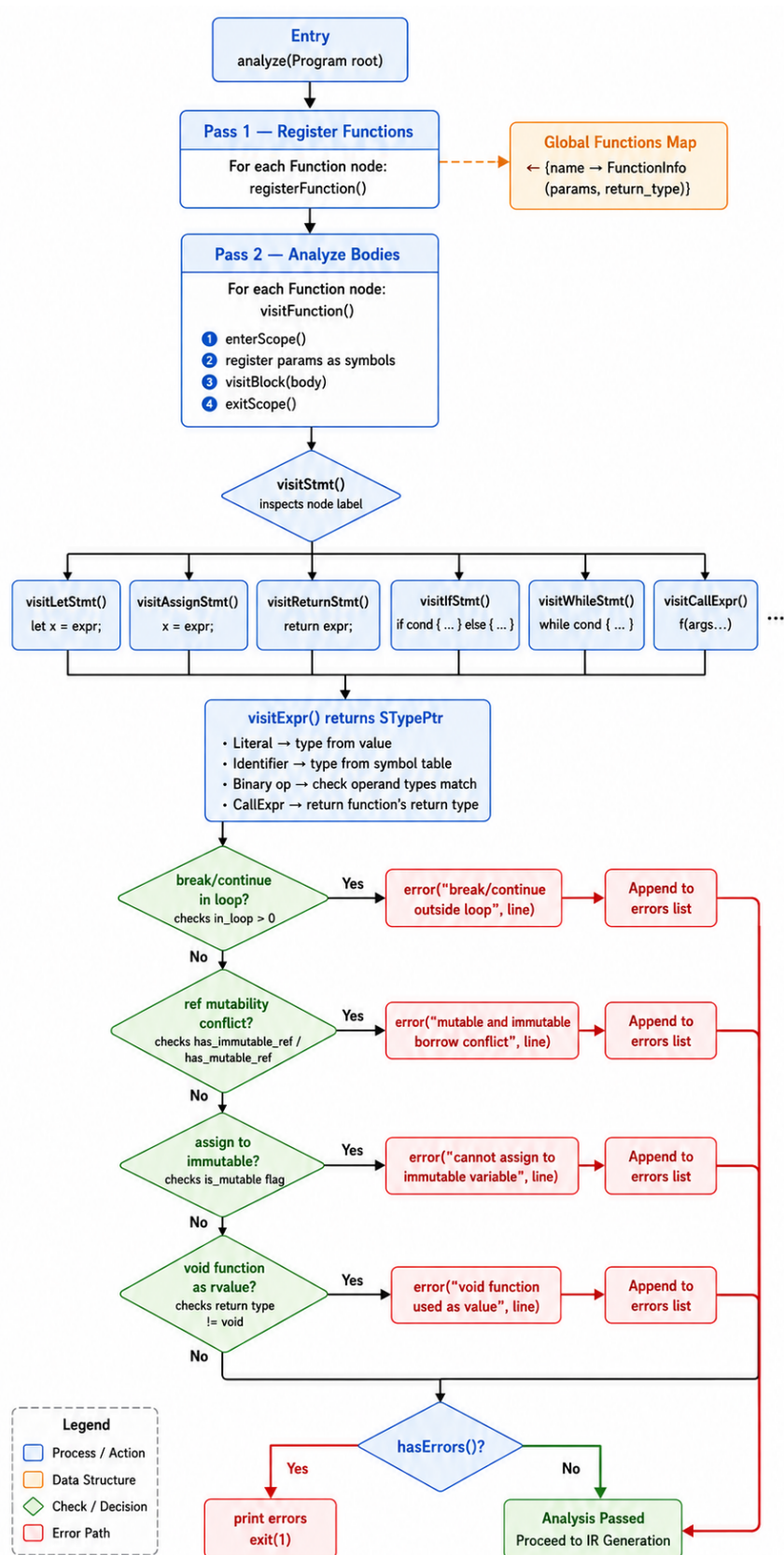


图 6: 语义分析整体流程图

- `visitBlock(body)`: 遍历语句序列
4. 每种语句:`visitStmt()` 根据节点标签(`LetStmt/AssignStmt/ReturnStmt/IfStmt/...`) 分发到对应的 `visit` 函数
 5. 每个表达式: 对应的 `visit` 函数返回 `STypePtr`, 沿树向上传播类型信息。类型信息从叶子节点(字面量、标识符)向上汇聚到根节点
 6. 在关键位置(赋值的 `=` 号处、函数调用的实参处、`return` 语句处)触发类型一致性检查
 7. 记录所有错误到内部的 `vector<SemanticError>` 列表, 不中断分析
 8. 分析完成后统一输出所有错误; 若存在任何错误, 终止后续阶段

3 中间代码生成器设计与实现

中间代码生成是编译过程的第四阶段，负责将经过语义检查的语法树转换为与目标机器无关的中间表示。本章详细介绍中间代码生成器的设计与实现。

3.1 中间代码理论基础

3.1.1 为什么需要中间代码

中间代码（Intermediate Representation, IR）是编译器前端和后端之间的桥梁：

- 与源语言解耦：不同高级语言可以生成相同的 IR，共享后端
- 与目标机器解耦：同一 IR 可以通过不同后端生成不同平台的机器码
- 便于优化：IR 形式简洁、语义明确，适合各种优化算法
- 便于理解和调试：比汇编更接近源语言，比语法树更接近机器

3.1.2 四元式（Quadruple）

本项目选择**四元式**（Quadruple）作为中间代码的形式。四元式是三地址码的一种具体表示，每条指令包含四个字段：

$$(op, arg1, arg2, result)$$

其中：

- **op**：操作码，指示要执行的操作
- **arg1**：第一操作数（变量名、临时变量、常量、标签）
- **arg2**：第二操作数（变量名、临时变量、常量、标签）
- **result**：结果存放位置（变量名或临时变量）

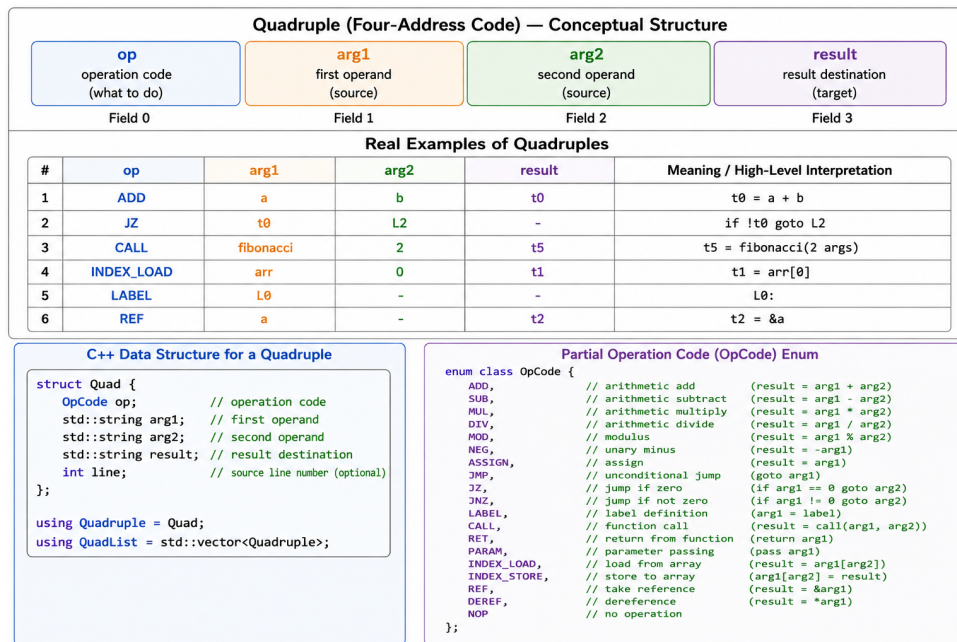


图 7: 四元式结构示意图

3.2 IR 操作码定义

表 6: IR 操作码 (IROp) 全集

分类	操作码	说明
算术运算	ADD	加法: $\text{result} = \text{arg1} + \text{arg2}$
	SUB	减法: $\text{result} = \text{arg1} - \text{arg2}$
	MUL	乘法: $\text{result} = \text{arg1} * \text{arg2}$
	DIV	除法: $\text{result} = \text{arg1} / \text{arg2}$
	NEG	取负: $\text{result} = -\text{arg1}$
比较运算	EQ	等于: $\text{result} = (\text{arg1} == \text{arg2})$
	NE	不等: $\text{result} = (\text{arg1} != \text{arg2})$
	LT	小于: $\text{result} = (\text{arg1} < \text{arg2})$
	LE	小于等于: $\text{result} = (\text{arg1} \leq \text{arg2})$
	GT	大于: $\text{result} = (\text{arg1} > \text{arg2})$
	GE	大于等于: $\text{result} = (\text{arg1} \geq \text{arg2})$
控制流	LABEL	标签定义 (跳转目标)
	JUMP	无条件跳转: <code>goto arg1</code>
	JZ	条件跳转: <code>if !arg1 goto arg2</code>
	JNZ	条件跳转: <code>if arg1 goto arg2</code>
	NOP	空操作
函数	FUNC_BEGIN	函数开始
	FUNC_END	函数结束
	PARAM	实参传递
	CALL	函数调用: $\text{result} = \text{arg1}(\text{arg2 个参数})$
	RETURN	函数返回
数据操作	ASSIGN	赋值: $\text{result} = \text{arg1}$
	INDEX_LOAD	数组读: $\text{result} = \text{arg1}[\text{arg2}]$
	INDEX_STORE	数组写: $\text{arg1}[\text{arg2}] = \text{result}$
	REF	取引用: $\text{result} = \&\text{arg1}$
	DEREF	解引用: $\text{result} = *\text{arg1}$
	TUPLE_GET	元组元素访问
	ARRAY_LIT	数组字面量填充
循环控制	BREAK	跳出循环
	CONTINUE	继续下一次循环

3.3 四元式数据结构

```

1 enum class IROp {
2     LABEL, ASSIGN, ADD, SUB, MUL, DIV,
3     EQ, NE, LT, LE, GT, GE,
4     JUMP, JZ, JNZ,
5     PARAM, CALL, RETURN,
6     INDEX_STORE, INDEX_LOAD,
7     REF, Deref,
8     NEG, NOP,
9     FUNC_BEGIN, FUNC_END,
10    TUPLE_GET, ARRAY_LIT,
11    BREAK, CONTINUE
12 };
13
14 struct Quadruple {
15     IROp op;
16     string arg1;    // 第一操作数（或标签名）
17     string arg2;    // 第二操作数
18     string result;  // 结果/目标
19
20     string toString() const {
21         // 格式: (OP, arg1, arg2, result)
22         // 空值用 "-" 表示
23     }
24 };

```

Listing 13: 四元式数据结构

3.4 IR 生成器核心实现

3.4.1 临时变量与标签管理

IR 生成器内部维护两个计数器，自动生成唯一的临时变量名和标签名：

```

1 class IRGenerator {
2     int temp_counter = 0;    // 临时变量计数器
3     int label_counter = 0;   // 标签计数器
4
5     string newTemp() { return "t" + to_string(temp_counter++); }
6     string newLabel() { return "L" + to_string(label_counter++); }
7

```

```

8   void emit(IROp op, const string& a1="", const string& a2="",
9           const string& r="") {
10      code.emplace_back(op, a1, a2, r);
11  }
12 };

```

Listing 14: 临时变量与标签管理

`newTemp()` 生成形如 `t0, t1, t2, ...` 的临时变量名, 用于持有表达式的中间计算结果。

`newLabel()` 生成形如 `L0, L1, L2, ...` 的标签名, 用于控制流语句 (`if/while/for/loop`) 的跳转目标。

3.4.2 表达式 IR 生成

表达式的 IR 生成采用与语法分析器相同的分层结构。每个表达式生成函数返回持有计算结果的临时变量名 (或变量名):

```

1 string genExpr(const Node* node) {
2     // 根据节点标签分发到对应的生成函数
3     if (lbl == "Literal")    return genLiteral(node);
4     if (lbl == "Identifier") return genAtom(node);           // 返回变量名
5     if (lbl == "CallExpr")   return genCallExpr(node);
6     if (lbl == "CmpExpr")    return genBinary(node, true);
7     if (lbl == "AddExpr")    return genBinary(node, false);
8     if (lbl == "MulExpr")    return genBinary(node, false);
9     if (lbl == "RefExpr")    return genRefExpr(node);
10    if (lbl == "DerefExpr")   return genDerefExpr(node);
11    if (lbl == "IndexExpr")   return genIndexExpr(node);
12    // ...
13 }
14
15 string genBinary(const Node* node, bool isCmp) {
16     // 1. 递归生成左操作数的 IR → left
17     // 2. 递归生成右操作数的 IR → right
18     // 3. 根据运算符选择操作码 (ADD/SUB/MUL/DIV/EQ/NE/...)
19     // 4. 生成临时变量 → result = newTemp()
20     // 5. emit(op, left, right, result)
21     // 6. return result
22 }

```

Listing 15: 表达式 IR 生成函数

示例：表达式 $a + b * c$ 的四元式生成过程：

```

1 // genBinary 处理 AddExpr:
2 //   left  = genExpr(a) → 返回 "a"
3 //   right = genBinary 处理 MulExpr:
4 //       left' = genExpr(b) → 返回 "b"
5 //       right' = genExpr(c) → 返回 "c"
6 //       result' = newTemp() → "t0"
7 //       emit(MUL, b, c, t0)
8 //       return "t0"
9 //   result = newTemp() → "t1"
10 //   emit(ADD, a, t0, t1)
11
12 // 最终生成:
13 // (MUL, b, c, t0)
14 // (ADD, a, t0, t1)

```

Listing 16: $a + b * c$ 的 IR 生成过程

3.4.3 控制流语句的 IR 生成

控制流语句的翻译是 IR 生成的核心难点。下面分别展示各控制流结构的翻译模式。

if 语句翻译：

```

1 void genIfStmt(const Node* node) {
2     string elseLabel = newLabel(); // else分支入口
3     string endLabel = newLabel(); // if语句出口
4
5     // Step 1: 生成条件表达式的IR → cond
6     string cond = genExpr(conditionNode);
7
8     // Step 2: 条件跳转: cond为假 → 跳到else
9     emit(JZ, cond, elseLabel);
10
11    // Step 3: 生成then块的IR
12    genBlock(thenBlock);
13
14    // Step 4: then块结束后跳过else
15    emit(JUMP, endLabel);
16
17    // Step 5: else入口标签

```

```

18     emit(LABEL, elseLabel);
19
20     // Step 6: 生成 else 块 (或 else-if) 的 IR
21     if (elseClause) { /* ... */ }
22
23     // Step 7: 出口标签
24     emit(LABEL, endLabel);
25 }

```

Listing 17: if 语句 IR 生成

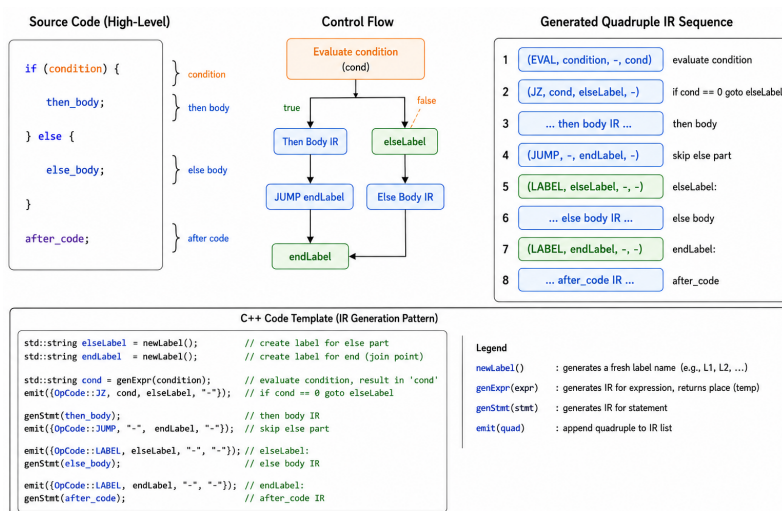


图 8: if 语句的四元式翻译模式

while 语句翻译:

```

1 void genWhileStmt(const Node* node) {
2     string startLabel = newLabel(); // 循环起始标签
3     string endLabel = newLabel();   // 循环出口标签
4
5     // Step 1: 循环起始标签
6     emit(LABEL, startLabel);
7
8     // Step 2: 生成条件 IR → cond
9     string cond = genExpr(conditionNode);
10
11    // Step 3: cond 为假 → 跳出循环
12    emit(JZ, cond, endLabel);
13
14    // Step 4: 设置 break(→endLabel) 和 continue(→startLabel)

```

```

15  string oldBreak = break_label;
16  string oldContinue = continue_label;
17  break_label = endLabel;
18  continue_label = startLabel;
19
20  // Step 5: 生成循环体的 IR
21  genBlock(bodyBlock);
22
23  // Step 6: 循环体结束 → 跳回起始
24  emit(JUMP, startLabel);
25
26  // Step 7: 出口标签
27  emit(LABEL, endLabel);
28
29  // 恢复外层循环的 break/continue 标签
30  break_label = oldBreak;
31  continue_label = oldContinue;
32 }

```

Listing 18: while 语句 IR 生成

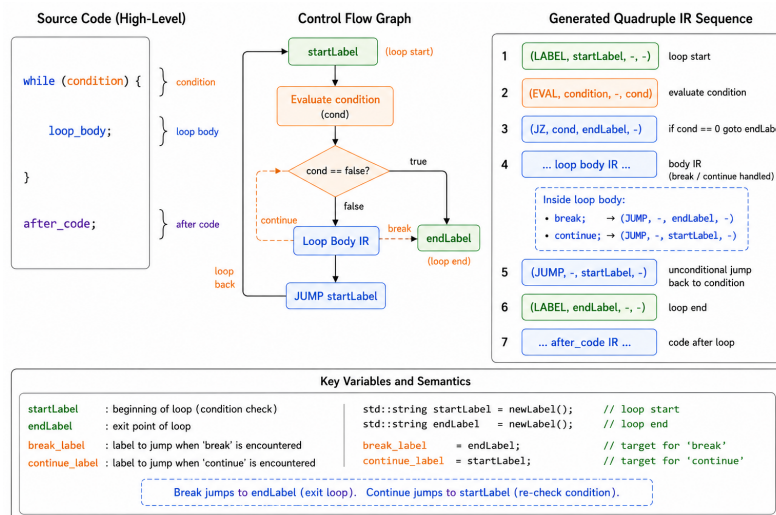


图 9: while 语句的四元式翻译模式

for 语句翻译:

for 循环翻译为 while 循环的等价形式。以 `for mut i in start..end { body }` 为例，翻译分为三个阶段：初始化 (`i=start`)、条件判断 (`i<end`)、递增 (`i=i+1`)。

```

1 // === for mut i in start..end { body } ===

```



```

2 // 阶段1: 循环初始化 (隐含的 i = start)
3 (ASSIGN, start, -, i)           // 因为for未显式写出, 由IR生成器补全
4 (LABEL, for_start, -, -)       // 循环起始标签
5 (LT, i, end, cond)             // 循环条件: i < end ?
6 (JZ, cond, for_end)           // 不满足则跳出
7 // ... body IR ...
8 // continue跳转目标:
9 (LABEL, for_step, -, -)
10 (ADD, i, 1, i)                // i = i + 1 (递增步长固定为1)
11 (JUMP, for_start, -, -)       // 跳回条件判断
12 (LABEL, for_end, -, -)       // 循环出口

```

Listing 19: for 语句的四元式翻译模式

注意 for 循环的 continue 标签指向 step 标签 (而非 start), 这确保 continue 会先执行递增操作再检查条件, 与 Rust 标准语义一致。

嵌套 if-else 的翻译: 当 if 语句包含 else-if 链时, 翻译模式递归展开。以 if a>0 {...} else if a<0 {...} else {...} 为例:

```

1 // 计算外层条件 a > 0 → t0
2 (GT, a, 0, t0)
3 (JZ, t0, else_if_label)       // a≤0 → 跳到else if
4 // then分支 (a>0)
5 ... then_body IR ...
6 (JUMP, end_label)             // 跳过所有else
7
8 (LABEL, else_if_label, -, -)
9 // 计算else-if条件 a < 0 → t1
10 (LT, a, 0, t1)
11 (JZ, t1, else_label)         // a≥0 → 跳到else
12 // else-if分支 (a<0)
13 ... else_if_body IR ...
14 (JUMP, end_label)
15
16 (LABEL, else_label, -, -)
17 // else分支 (a==0)
18 ... else_body IR ...
19
20 (LABEL, end_label, -, -)

```

Listing 20: 嵌套 if-else 的四元式翻译

这种翻译模式保证了三个分支的互斥执行：通过连续的 JZ 条件跳转链，每个分支的末尾以无条件 JUMP 跳过后续分支，形成清晰的 if-else if-else 控制流结构。

loop 语句翻译：

loop 是无限循环，只有显式的 break 能退出：

```

1 (LABEL, loop_start, -, -)
2 // ... body IR ...
3 // break → (JUMP, loop_end)
4 // continue → (JUMP, loop_start)
5 (JUMP, loop_start, -, -)
6 (LABEL, loop_end, -, -)

```

Listing 21: loop 语句的四元式翻译模式

嵌套循环的 break/continue 处理：

通过保存和恢复 break_label 和 continue_label 实现嵌套循环的支持：

```

1 // 进入内层循环时保存外层标签
2 string oldBreak = break_label;
3 string oldContinue = continue_label;
4 break_label = endLabel;          // 内层 break 跳到内层出口
5 continue_label = startLabel;    // 内层 continue 跳到内层起始
6
7 // ... 生成内层循环体 IR ...
8
9 // 离开内层循环时恢复外层标签
10 break_label = oldBreak;
11 continue_label = oldContinue;

```

Listing 22: 嵌套循环的 break/continue 标签管理

3.4.4 函数调用的 IR 生成

函数调用涉及参数传递和返回值处理：

```

1 string genCallExpr(const Node* node) {
2     string funcName = extractId(node);
3
4     // Step 1: 为每个实参生成 PARAM 指令
5     for (每个实参) {
6         string argVal = genExpr(实参);

```

```

7      emit(PARAM, argVal);
8  }
9
10     // Step 2: 生成 CALL 指令
11     string result = newTemp();
12     if (函数有返回值) {
13         emit(CALL, funcName, 参数数量, result);
14         return result;    // 返回临时变量持有函数结果
15     } else {
16         emit(CALL, funcName, 参数数量, "");
17         return "";        // 无返回值
18     }
19 }

```

Listing 23: 函数调用 IR 生成

例如, 调用 `add(x, y)` 生成的四元式:

```

1 (PARAM, x, -, -)
2 (PARAM, y, -, -)
3 (CALL, add, 2, t5)

```

3.4.5 引用与解引用的 IR 生成

```

1 string genRefExpr(const Node* node) {
2     string inner = genExpr(innerExpr);    // 被引用变量
3     string result = newTemp();
4     emit(REF, inner, "", result);        // result = &inner
5     return result;
6 }
7
8 string genDerefExpr(const Node* node) {
9     string inner = genExpr(innerExpr);    // 引用变量
10    string result = newTemp();
11    emit(DEREF, inner, "", result);        // result = *inner
12    return result;
13 }

```

Listing 24: 引用与解引用的 IR 生成

3.5 完整的 IR 生成示例

以 fibonacci 函数为例，展示完整程序的 IR 输出，并对每一段 IR 进行逐行解析：

```

1 fn fibonacci(mut n:i32) -> i32 {
2     if n<=1 { return n; }           // 基线条件
3     let mut a:i32=0;                // fib(0)
4     let mut b:i32=1;                // fib(1)
5     let mut i:i32=2;                // 循环计数器从2开始
6     while i<=n {                    // 迭代计算
7         let mut t:i32=a+b;          // t = fib(i)
8         a=b;                        // fib(i-2) + fib(i-1)
9         b=t;                        // fib(i-1) + fib(i)
10        i=i+1;
11    }
12    return b;                        // 返回 fib(n)
13 }
```

Listing 25: fibonacci 函数源码（13 行有效代码）

该函数生成的 IR 共 23 条四元式：

```

1 (0) (FUNC_BEGIN, fibonacci, -, -) // 函数入口标记
2 (1) (ASSIGN, param_n, -, n)       // 接收参数: n = input
3 // === if n<=1 条件判断 ===
4 (2) (LE, n, 1, t0)                // t0 = (n <=
    1), 比较运算结果存入临时变量
5 (3) (JZ, t0, L0, -)               // 若 t0 为假(n>1), 跳转到
    L0 (跳过return)
6 // === then分支: return n ===
7 (4) (RETURN, n, -, -)             // n<=1时直接返回n
    (fib(0)=0, fib(1)=1)
8 (5) (LABEL, L0, -, -)             // 分支出口标签 (n>1时执行)
9 // === 初始化循环变量 ===
10 (6) (ASSIGN, 0, -, a)             // a = 0 (fib(0))
11 (7) (ASSIGN, 1, -, b)             // b = 1 (fib(1))
12 (8) (ASSIGN, 2, -, i)             // i = 2 (从第2项开始)
13 // === while i<=n 循环 ===
14 (9) (LABEL, L1, -, -)             // while 起始标签 (循环入口)
15 (10) (LE, i, n, t1)               // t1 = (i <= n) 循环条件判断
16 (11) (JZ, t1, L2, -)              // 条件为假, 跳出循环到 L2
17 // === 循环体: 斐波那契迭代 ===
```

```

18 (12) (ADD, a, b, t2) // t2 = a + b
    (当前斐波那契数)
19 (13) (ASSIGN, t2, -, t) // t = t2
20 (14) (ASSIGN, b, -, a) // a = b (右移: fib(k-2) ←
    fib(k-1))
21 (15) (ASSIGN, t, -, b) // b = t (右移: fib(k-1) ←
    fib(k))
22 (16) (ADD, i, 1, t3) // t3 = i + 1
23 (17) (ASSIGN, t3, -, i) // i = t3 (计数器递增)
24 (18) (JUMP, L1, -, -) // 无条件跳回循环起始 L1
25 // === 循环出口 ===
26 (19) (LABEL, L2, -, -) // while 出口标签
27 (20) (RETURN, b, -, -) // 返回 fib(n) = b
28 // === 函数收尾 ===
29 (21) (LABEL, func_fibonacci_end, -, -) // 函数结束标签 (统一出口)
30 (22) (FUNC_END, fibonacci, -, -) // 函数结束标记

```

Listing 26: fibonacci 函数的四元式 IR (含逐行注释)

IR 生成分析：这 23 条 IR 展示了几个关键的翻译模式：

- **参数传递**（行 1）：函数参数通过特殊的 `param_n` 形式映射到局部变量 `n`
- **条件分支**（行 2-5）：if 语句的标准模式——计算条件 → 条件跳转 → then 体 → 标签出口
- **循环翻译**（行 9-18）：while 的标准模式——入口标签 → 条件判断 → JZ 跳出 → 循环体 → 无条件跳回 → 出口标签
- **表达式分解**（行 12-17）：每个子表达式产生临时变量 (`t2`, `t3`)，赋值使用独立的 `ASSIGN` 指令
- **return 收尾**（行 21）：统一在函数末尾设置 `func_name_end` 标签，return 语句在产生 `RETURN` 后也会跳转到此标签（函数内没有其他代码时省略 `JUMP`）

从 13 行源码生成 23 条 IR，平均每行源码约 1.8 条 IR，这个比例反映了我们设计的 IR 粒度——既不冗余也不过粗。

3.6 IR 生成流程总览

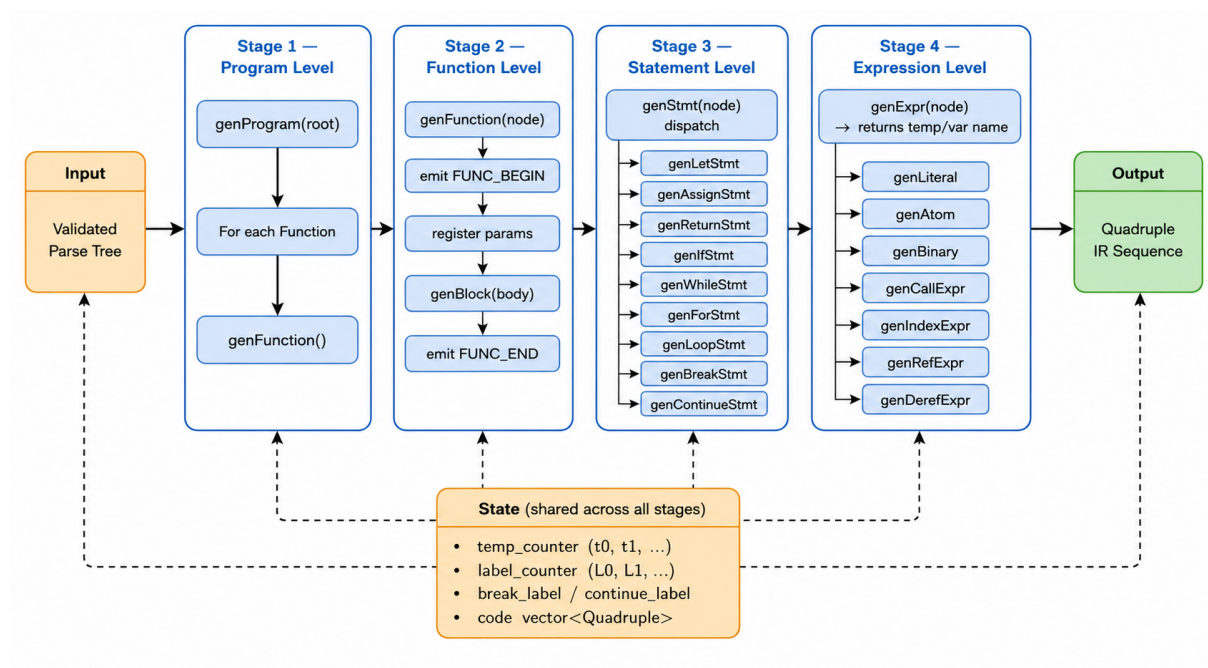


图 10: 中间代码生成整体流程图

4 测试与结果分析

本章通过完整的测试用例验证语义分析和中间代码生成的正确性。

4.1 测试策略

测试采用**集成测试策略**：完整的编译流水线（词法 → 语法 → 语义 → IR）验证端到端正确性：

1. **正向测试**（Pass tests）：编写语法和语义均正确的代码，验证分析通过并生成 IR
2. **负向测试**（Fail tests）：编写包含特定语义错误的代码，验证错误被正确检测
3. **全规则覆盖**：确保每条作业要求中的语义规则都有对应的测试用例

4.2 测试用例设计

4.2.1 综合正确用例：test_full.rs

该文件包含 30 余个函数，覆盖全部必做规则（0.1–5.1）和选做规则（4.2–9.2），是语义分析的完整正确性验证：

```
1 // 1.5 函数输出
2 fn test_1_5() -> i32 {
3     return 1;
4 }
5
6 // 2.3 变量声明赋值（含 shadowing）
7 fn test_2_3() {
8     let mut a:i32=1;
9     let mut b=2;
10    let mut a=3;           // shadowing
11 }
12
13 // 6.3 可变引用
14 fn test_6_3(mut a:i32) {
15     let mut b:&mut i32=&mut a;
16 }
17
18 // 综合用例：fibonacci
```

```

19 fn fibonacci(mut n:i32) -> i32 {
20     if n<=1 { return n; }
21     let mut a:i32=0;
22     let mut b:i32=1;
23     let mut i:i32=2;
24     while i<=n {
25         let mut t:i32=a+b;
26         a=b; b=t; i=i+1;
27     }
28     return b;
29 }

```

Listing 27: test_full.rs 中的关键函数示例

测试结果: test_full.rs 通过语义分析, 无错误报告, 并成功生成 97 行四元式 IR。

4.2.2 语义错误检测用例: test_errors.rs

该文件包含 17 个函数, 每个函数恰好触发一类语义错误:

```

1 // 错误1,2: 返回类型不一致
2 fn err_1_5a() -> i32 { return ; }           // void->i32
3 fn err_1_5b() { return 1; }                 // i32->void
4
5 // 错误3: 赋值给未声明变量
6 fn err_2_2a() { a=32; }
7
8 // 错误6: 不可变变量二次赋值
9 fn err_6_1() { let c:i32=1; c=2; }
10
11 // 错误13: 可变引用与不可变引用共存
12 fn err_6_3a() {
13     let mut a:i32=1;
14     let b=&a; let mut c=&mut a;
15 }
16
17 // 错误17: 数组初始化元素数量不一致
18 fn err_8_2a(mut a:i32) {
19     let mut a:[i32;2];
20     a=[1,2,3];
21 }

```


Listing 28: test_errors.rs 语义错误测试

测试结果：test_errors.rs 中的全部 17 个函数均被正确检测出语义错误，每个函数精确产生一类错误信息。以下是实际运行时 parser 输出的部分错误信息：

```

1 === Semantic Errors ===
2 Error: return statement type (void) does not match function
3     return type (i32) at line 2
4 Error: return statement type (i32) does not match function
5     return type (void) at line 6
6 Error: variable 'a' not declared at line 10
7 Error: type mismatch in assignment: i32 vs bool at line 14
8 Error: variable 'a' not declared at line 18
9 Error: variable 'a' used before assignment at line 23
10 Error: cannot assign to immutable variable 'c' at line 29
11 Error: function 'err_3_5a_helper' expects 0 argument(s)
12     but got 1 at line 34
13 Error: cannot use void expression as initializer at line 40
14 Error: argument 1 type mismatch in call to
15     'err_3_5c_helper': expected i32 but got bool at line 46
16 Error: break statement must be inside a loop at line 50
17 Error: continue statement must be inside a loop at line 54
18 Error: cannot create mutable reference to 'a':
19     other references already exist at line 60
20 Error: cannot create mutable reference to
21     immutable variable 'a' at line 66
22 Error: cannot dereference non-reference type: i32 at line 71
23 Error: cannot assign through immutable reference at line 77
24 Error: type mismatch in assignment: [i32;2] vs [i32;3] at line 82
25 Error: type mismatch in assignment: (i32,i32) vs (i32,i32,i32)
26     at line 87

```

Listing 29: 语义错误检测实际输出（节选）

每条错误信息都包含**错误描述**和**行号定位**，便于开发者快速定位问题。错误信息中的类型名称使用可读的字符串表示（如 [i32;2]、(i32,i32)），而非内部枚举编号。

4.3 批量测试脚本与结果

```

1 #!/bin/bash
2 PARSER="./build/bin/parser"
3 PASS=0; FAIL=0; TOTAL=0
4

```

```

5 run_test() {
6     # 运行 parser, 检查 exit code
7     # expect_pass="pass" 时期望 exit code=0
8     # expect_pass="fail" 时期望 exit code 0
9 }
10
11 # Rule 1.x: 程序结构 (7 tests)
12 # Rule 2.x: 变量声明与赋值 (4 tests)
13 # Rule 3.x: 表达式 (2 tests)
14 # Rule 3.5: 函数调用 (2 tests)
15 # Rule 4.x: 选择结构 (1 test)
16 # Rule 5.x: 循环 (3 tests)
17 # Rule 6.x: 引用 (2 tests)
18 # Rule 8.x: 数组 (1 test)
19 # Rule 9.x: 元组 (1 test)
20 # Total: 22 tests

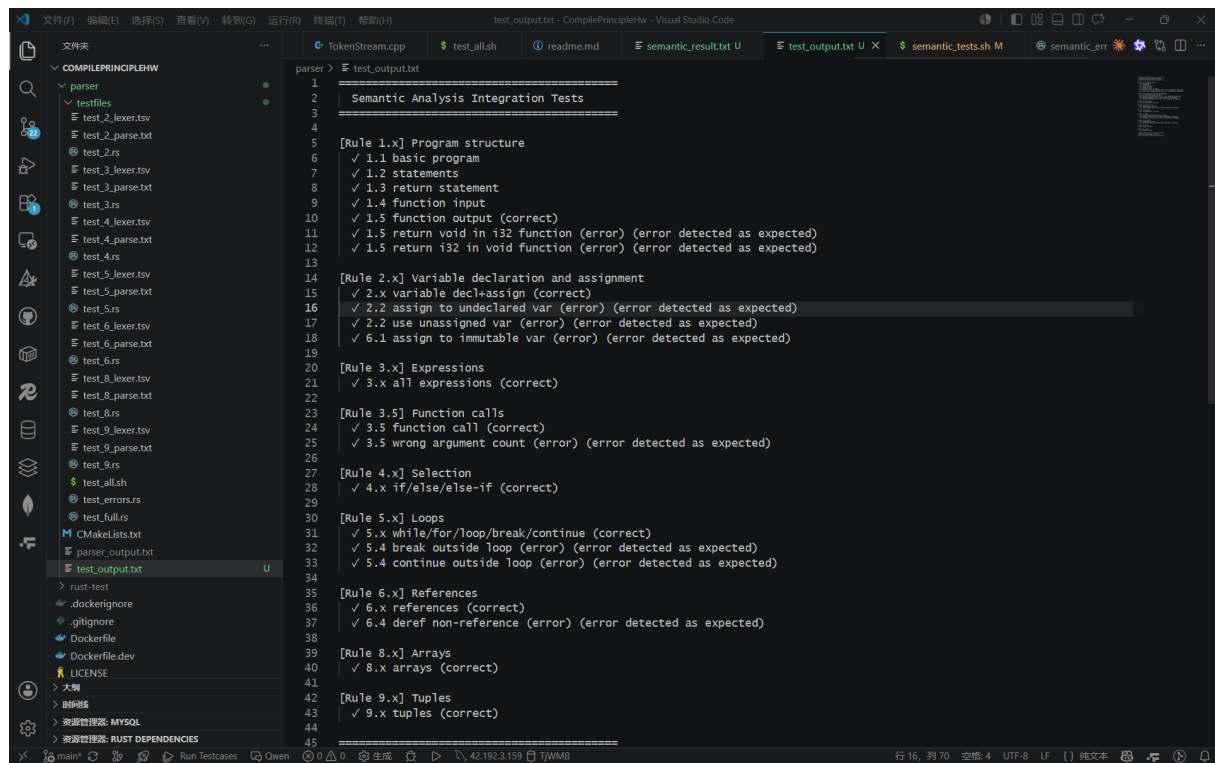
```

Listing 30: semantic_tests.sh 测试脚本结构

表 7: 测试结果汇总

测试规则范围	测试数量	通过	结果
规则 1.x (程序结构)	7	7	✓
规则 2.x (变量声明与赋值)	4	4	✓
规则 3.x (表达式)	2	2	✓
规则 3.5 (函数调用)	2	2	✓
规则 4.x (选择结构)	1	1	✓
规则 5.x (循环)	3	3	✓
规则 6.x (引用)	2	2	✓
规则 8.x (数组)	1	1	✓
规则 9.x (元组)	1	1	✓
合计	22	22	100%

4.4 运行效果



```
1 =====
2 Semantic Analysis Integration Tests
3 =====
4
5 [Rule 1.x] Program structure
6   ✓ 1.1 basic program
7   ✓ 1.2 statements
8   ✓ 1.3 return statement
9   ✓ 1.4 function input
10  ✓ 1.5 function output (correct)
11  ✓ 1.5 return void in i32 function (error) (error detected as expected)
12  ✓ 1.5 return i32 in void function (error) (error detected as expected)
13
14 [Rule 2.x] Variable declaration and assignment
15   ✓ 2.x variable decl+assign (correct)
16   ✓ 2.2 assign to undeclared var (error) (error detected as expected)
17   ✓ 2.2 use unassigned var (error) (error detected as expected)
18   ✓ 6.1 assign to immutable var (error) (error detected as expected)
19
20 [Rule 3.x] Expressions
21   ✓ 3.x all expressions (correct)
22
23 [Rule 3.5] Function calls
24   ✓ 3.5 function call (correct)
25   ✓ 3.5 wrong argument count (error) (error detected as expected)
26
27 [Rule 4.x] Selection
28   ✓ 4.x if/else/else-if (correct)
29
30 [Rule 5.x] Loops
31   ✓ 5.x while/for/loop/break/continue (correct)
32   ✓ 5.4 break outside loop (error) (error detected as expected)
33   ✓ 5.4 continue outside loop (error) (error detected as expected)
34
35 [Rule 6.x] References
36   ✓ 6.x references (correct)
37   ✓ 6.4 deref non-reference (error) (error detected as expected)
38
39 [Rule 8.x] Arrays
40   ✓ 8.x arrays (correct)
41
42 [Rule 9.x] Tuples
43   ✓ 9.x tuples (correct)
44
45 =====
```

图 11: 命令行语义分析测试运行截图（22 项测试全部通过）

5 思考与分析

5.1 语义分析中 Visitor 模式的设计考量

5.1.1 为什么选择 Visitor 模式

在实现语义分析时，我们面临两种主要的设计选择：

1. 在语法树节点中直接添加分析方法：在 Node 类中添加 virtual analyze 方法
2. 使用外部 Visitor 类：将语义分析逻辑集中到独立的 SemanticAnalyzer 类中

我们选择了方案 2（Visitor 模式），主要理由：

- 关注点分离：语法树节点仅负责存储结构，分析与生成逻辑独立
- 可扩展性：可以方便地添加新的分析 Pass（如后续的优化 Pass）而不修改 Node 类
- Info passing：表达式 Visitor 可以返回值（类型信息），语句 Visitor 无需返回值，这种差异在 Visitor 模式中自然表达

5.1.2 表达式类型传播的设计

在实现中，所有 visitXxxExpr 方法返回 STypePtr，沿语法树自底向上传播类型信息。这种设计的优势：

- 表达式类型推导自然：字面量 → 基本类型，运算符 → 结果类型，标识符 → 查符号表
- 类型检查位置明确：在赋值、传参、返回等节点比较左右类型
- 代码可读性高：每个 visit 方法的返回值直接表达“这个子树求值后的类型”

5.2 两遍扫描 vs 单遍扫描的权衡

处理函数前向引用有两个常见策略：

1. 两遍扫描：先收集所有声明（函数签名、类型定义），再分析函数体

2. **单遍扫描 + 待决列表**: 遇到未定义的函数调用时, 记录到待决列表, 待函数定义后回填

我们选择了方案 1, 原因如下:

- **实现简单**: 两遍扫描逻辑清晰, 不需要处理复杂的待决回填
- **语义正确**: 在我们的语言中, 函数声明位于顶层, 不存在嵌套函数, 两遍扫描天然正确
- **与 Rust 行为一致**: Rust 允许函数在任何位置定义, 不做前向声明要求

方案 1 的代价是额外的语法树遍历 (第一遍注册签名), 但由于语法树已在内存中, 第二遍遍历的开销可以接受。

5.3 借用追踪的简化实现与局限

本项目的借用追踪使用了**标志位方法** (`has_immutable_ref` / `has_mutable_ref`), 在每个变量的符号表条目中记录当前存在的引用类型。这种实现能够检测作业要求的所有借用错误场景:

- 可变引用与其他引用共存 → 检测到 `has_immutable_ref==true`
- 从不可变变量创建可变引用 → 检测到 `is_mutable==false`
- 不可变引用修改数据 → 在赋值左侧检测 `ref` 类型

但与 Rust 编译器完整的**借用检查器** (Borrow Checker, 基于 NLL 非词法生命周期) 相比, 我们的实现有以下局限:

1. **未追踪引用的生命周期**: 标志位一旦设置就不会清除, 即使引用已经超出作用域
2. **未处理函数间的借用**: 函数参数和返回值的借用关系未做跨函数分析
3. **简化了所有权模型**: 我们的语言只支持 `i32` 类型 (Copy 语义), 不涉及 `move` 语义和所有权转移

这些简化是合理的——作业要求的语言子集不需要完整的 Rust 借用检查器, 当前的实现完全覆盖了作业中的借用规则检测需求。

5.4 四元式 vs 其他 IR 形式的对比

我们选择四元式作为 IR 形式，与其他常见 IR 形式对比：

表 8: IR 形式对比

IR 形式	表示	优点	缺点
四元式	(op, a1, a2, r)	实现简单，输出清晰	临时变量多，优化需额外处理
三元式	(op, a1, a2)	无临时变量	移动代码时需更新引用
间接三元式	(op, a1, a2)+ 间接表	优化友好	实现复杂
SSA 形式	基于 函数	优化最自然	实现复杂，对课程要求过重

四元式是编译原理教材中最经典的形式，适合教学演示。每条指令的结果有独立的存储位置（临时变量），便于后续优化 Pass（如公共子表达式消除、常量传播）对指令进行重排序和删除。

5.5 控制流翻译中标签管理的设计

控制流翻译的核心是**标签管理**。我们的设计中，break 和 continue 的跳转目标通过两个变量（break_label, continue_label）存储，在进入嵌套循环时保存/恢复：

```

1 // 伪代码：进入内层循环时
2 string oldBreak = break_label;      // 保存外层 break 目标
3 break_label = innerEndLabel;        // 内层 break 指向内层循环出口
4
5 genBlock(innerBody);
6
7 break_label = oldBreak;              // 恢复外层 break 目标

```

Listing 31: 嵌套循环的标签栈管理

这种**单层保存策略**（而非完整的标签栈）之所以足够，是因为我们的程序设计使得每次进入循环体时就确定了 break/continue 目标，离开循环体时就恢复——本质上是一个隐式栈，通过函数调用栈来实现。这种设计简洁且在正确的递归生成 IR 代码时行为正确。

5.6 语义错误的组织与报告策略

语义分析器的错误处理采用**先收集、后报告**的策略：

1. **收集阶段：**所有 `error()` 调用将错误信息追加到内部 `vector<SemanticError>`，而非立即抛出异常。这使得语义分析可以继续运行，检测更多错误
2. **报告阶段：**分析完成后，由 `main.cpp` 调用 `analyzer.hasErrors()` 判断是否存在任何错误。如有错误，调用 `analyzer.printErrors()` 统一输出
3. **阻断策略：**一旦存在语义错误，IR 生成阶段被跳过（`main.cpp` 中 `return 1`）。这是因为在存在语义错误的程序上生成 IR 可能导致后续阶段的行为未定义

这种策略与工业编译器（如 GCC 的 `-Werror`、Rust 的 `rustc`）一致：尽可能多地发现错误，一次性报告给用户，避免“修复一个错误又出现另一个”的低效循环。不过，当前实现的一个限制是：遇到错误后某些后续检查可能被跳过（如符号表条目缺失导致后续访问该符号的检查无法进行），这是**错误恢复**（Error Recovery）能力的不足，可作为改进方向。

5.7 类型系统的设计哲学：结构化 vs 命名化

我们在类型系统中做的一个关键设计选择是采用**结构化类型等价**（Structural Type Equivalence）而非**命名类型等价**（Nominal Type Equivalence）。具体体现在：

- 两个 `[i32; 3]` 类型总是等价，无论它们声明在何处
- 两个 `(i32, i32)` 元组类型总是等价
- 类型没有“名字”概念——比较的是类型的**内部结构**而非**外部名称**

这种选择是合理的，因为我们的语言没有 `type` 别名或用户自定义类型（如 `struct`），所有类型都是内建的组合类型。如果未来扩展支持用户自定义类型，可能需要引入命名等价的概念来区分“结构相同但语义不同”的类型。

5.8 IR 操作码设计中的冗余与完备性

回顾 27 种 IR 操作码的设计，可以发现一些有趣的设计模式：

- **对称性：**算术运算（`ADD/SUB/MUL/DIV`）和比较运算（`EQ/NE/LT/LE/GT/GE`）各包含一组完备的操作。六个比较运算符构成了完整的偏序关系集合
- **最小完备性：**理论上 `JZ` 和 `JUMP` 就足以表达所有控制流（`JNZ = JZ` 的取反、循环和分支可用 `JZ+JUMP` 实现），但我们仍然提供了 `JNZ` 以简化某些场景的 IR 生成（如逻辑与的短路求值）。`BREAK` 和 `CONTINUE` 也可以归约为 `JUMP`，但保留它们使 IR 更具可读性

- **数据操作层次：**INDEX_LOAD/STORE、REF/DEREF 构成了引用和复合类型操作的基础。ARRAY_LIT 的引入是为了统一处理数组字面量初始化——每个元素调用一次 ARRAY_LIT 而非生成 N 条 ASSIGN，这样既清晰又为后续的数组初始化优化留出了空间

IR 设计的艺术在于在”太少”和”太多”之间找到平衡：太少则 IR 表达能力不足；太多则后续优化和代码生成需要处理过多 case。我们的 27 种操作码是经过对语言特性的仔细分析后确定的——每个操作码都有清晰的语义和独立的存在价值。

6 总结与反思

6.1 项目成果总结

本项目在上一阶段词法分析和语法分析的基础上，成功实现了语义分析器和中间代码生成器，完成了从源代码到中间代码的完整编译前端流水线。主要成果包括：

1. **语义分析器**：基于 Visitor 模式实现了完整的类型检查和语义错误诊断，覆盖 18 类语义错误
2. **类型系统**：设计了支持 7 种基础类型的类型层次结构，支持类型推导和类型一致性判断
3. **符号表**：实现了栈式作用域管理，支持变量 shadowing 和借用追踪
4. **中间代码生成器**：实现了四元式形式的 IR 生成，覆盖 27 种操作码
5. **控制流翻译**：正确翻译 if/else/while/for/loop/break/continue 等控制流结构
6. **完整测试**：22 项集成测试全部通过，覆盖全部必做 + 选做规则

6.2 遇到的困难与解决方案

1. 函数前向引用问题

问题：如果函数 A 调用了后面定义的函数 B，单遍扫描时 B 的签名尚未注册，导致“函数未声明”的误报。

解决方案：采用两遍扫描策略。第一遍遍历所有函数头声明，注册签名到全局函数表；第二遍再分析函数体。这种方法简单有效，正确处理了前向引用。

2. 类型系统的递归结构

问题：类型可以是嵌套的，如 `[[i32;3];3]`（数组的数组）、`&mut [i32;3]`（可变引用指向数组）。类型解析需要递归处理。

解决方案：使用 `shared_ptr<SType>` 实现类型的安全共享和递归引用。`parseTypeFromNode()` 方法递归解析嵌套类型，`inner` 指针指向内部类型。

3. 借用追踪的标志位清除

问题：在最初的实现中，借用标志位设置后不会清除，导致后续对同一变量的合法操作被误报为错误。

解决方案：调整了错误检测的触发条件，确保只在“实际”创建引用时已有冲突引用存在时才报错，而非在后续操作中报错。

4. 四元式的 arg2 参数处理

问题：不同操作码的 arg2 含义不同——JUMP 的 arg1 是目标标签，JZ 的 arg2 是目标标签，CALL 的 arg2 是参数数量。容易混淆。

解决方案：建立明确的约定文档：JUMP→arg1 为目标；JZ/JNZ→arg1 为条件，arg2 为目标；CALL→arg1 为函数名，arg2 为参数个数。在 toString() 中空值统一输出“-”。

6.3 知识收获与能力提升

通过本次项目实践，我们获得了以下收获：

1. **语义分析的完整理解：**从符号表设计到类型检查算法，深入理解了编译器“理解程序含义”的过程
2. **Visitor 模式的实际应用：**体会了设计模式在编译器构造中的价值——分离数据与操作
3. **中间代码的设计思维：**理解了四元式作为“通用汇编”的设计哲学，以及控制流结构的标准翻译模式
4. **测试驱动开发：**通过错误测试用例驱动语义分析的开发，每增加一个错误检测就增加对应的测试用例
5. **工程迭代能力：**在已有代码基础上的扩展能力——理解既有架构，在不破坏已有功能的前提下添加新功能

6.4 流水线编排与模块协作

整个编译流水线的编排在 main.cpp 中完成，形成了清晰的分阶段处理流程：

```
1 // 阶段1: 词法分析 → Token序列
2 TokenStream ts = lex(input);
3 ts.remove_comments();
4
5 // 阶段2: 语法分析 → 语法树
6 Parser parser(ts);
7 NodePtr tree = parser.parse();
8
9 // 阶段3: 语义分析 → 错误列表 OR 验证通过
10 SemanticAnalyzer analyzer;
```

```
11 analyzer.analyze(tree.get());
12 if (analyzer.hasErrors()) {
13     analyzer.printErrors(cout);
14     return 1; // 语义错误，终止后续阶段
15 }
16
17 // 阶段4：中间代码生成 → 四元式序列
18 IRGenerator irgen;
19 irgen.generate(tree.get());
20 irgen.writeIR(ir_output);
```

Listing 32: main.cpp 中的编译流水线编排

这种编排体现了编译器设计的经典原则：**每个阶段只做一件事，输入和输出明确**。阶段之间通过 `NodePtr`（语法树）作为唯一的数据桥梁——语法树是词法分析和语法分析的输出，也是语义分析和 IR 生成的输入。这种设计使得各阶段可以独立开发、独立测试。

6.5 改进方向与展望

项目仍有改进空间：

1. **IR 优化**：当前生成的 IR 未经过优化，存在大量冗余的临时变量和无用赋值。可添加基本块划分、死代码消除、常量折叠等优化 Pass。例如，`a = 0; b = a;` 当前生成两条 IR，但可优化为 `a = 0; b = 0;`（常量传播），或将连续的 ASSIGN 链合并
2. **更丰富的类型系统**：可扩展支持 `bool`、`f64`、字符串等基础类型。当前比较运算返回 `ST_BOOL`，但 `bool` 类型不能作为变量类型声明（声明 `bool` 需通过 `let b = 1==1` 的类型推导），添加原生 `bool` 类型将使语言更完整
3. **更严谨的借用检查**：引入 NLL（Non-Lexical Lifetime）风格的借用检查，替代当前的简单标志位方法。这需要追踪每个引用的“活跃区间”（liveness range），而非简单的 `bool` 标志
4. **目标代码生成**：将四元式 IR 进一步翻译为 x86 汇编或 LLVM IR，完成完整的编译后端。四元式 → 汇编的翻译是经典的后端任务，涉及寄存器分配、指令选择、调用约定等
5. **错误恢复**：语义分析器在遇到第一个错误后可继续分析（而非立即停止），报告所有语义错误。这需要错误恢复机制——例如，当符号表查不到变量时，不立即终止

当前函数分析，而是假设一个“错误类型”继续分析，避免级联错误

6. 优化 IR 的标签生成：当前每个 if/while 都会生成一对标签（L0/L1, L2/L3...），即使某些标签最终未被引用（如没有 else 的 if 语句，elseLabel 不会被作为跳转目标）。可添加标签清理 Pass，移除无用的 LABEL 指令

6.6 个人想法与体会

6.6.1 对语义分析在编译器中地位的认识

实现语义分析的过程让我们深刻体会到：语法正确 程序正确。一个程序即使完全符合语法规则，也可能存在大量错误——类型不匹配、变量未声明、违反借用规则等。语义分析是编译器“理解”程序的第一步，也是后续正确生成代码的基础。

在我们的实现中，有一个重要的设计原则：**语义分析不通过，不生成 IR**。即 main.cpp 中先运行语义分析，只有 `!analyzer.hasErrors()` 时才进入 IR 生成阶段。这与工业编译器（如 Rust 的 `rustc`）的行为一致——类型检查失败会阻止后续阶段。

6.6.2 对四元式设计的反思

四元式设计的一个关键权衡是**操作的粒度**。操作码越细粒度（如将 `a=b+c` 分解为明确的 `ADD+ASSIGN`），IR 越接近机器指令，后续优化和代码生成越容易；但 IR 的条数也会急剧增加。

我们选择了适中的粒度：二元运算保留在一条四元式中（`(ADD, b, c, t0)`），而赋值需要额外的 `ASSIGN`（`(ASSIGN, t0, -, a)`）。这比“一条 IR 完成 `a=b+c`”更清晰，比“分解为取 b、取 c、加、存 a 四条 IR”更简洁。这是一个在表达能力与 IR 规模之间的合理折中。

6.6.3 对渐进式文法设计的理解

本项目的文法设计采用典型的**分层拓展**方式：从最简单的空函数开始，逐层添加语句、表达式、控制流、引用、数组、元组。在实现语义分析时，这种分层结构的优势更加明显——每增加一层语法规则，对应的语义检查也可以分层添加。

例如，实现完基础表达式（3.1–3.4）的语义分析后，添加函数调用（3.5）只需要：

1. 在 `visitExpr` 中添加 `CallExpr` 分支
2. 实现 `visitCallExpr`：查函数表 → 检查实参数量 → 逐个检查实参类型 → 返回函数的返回类型

3. 添加对应的语义错误测试用例

这种增量式的开发方式降低了复杂度，使得为一个已有语法正确性验证的系统添加语义分析成为可控的过程。

7 参考文献

1. Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman. 编译原理（第 2 版）. 机械工业出版社, 2009.
2. Andrew W. Appel. Modern Compiler Implementation in C. Cambridge University Press, 1998.
3. Rust 程序设计语言（中文版）. <https://rustwiki.org/zh-CN/book/>
4. Rust Reference 一借用检查. <https://doc.rust-lang.org/reference/>
5. 北京大学编译实践课程在线文档. <https://pku-minic.github.io/online-doc/>
6. 自己动手写编译器. <https://pandolia.net/tinyc/index.html>
7. LLVM Language Reference Manual. <https://llvm.org/docs/LangRef.html>
8. CLIUtils/CLI11: Command line parser for C++11. <https://github.com/CLIUtils/CLI11>
9. antlr/grammars-v4: Rust grammar for ANTLR v4. <https://github.com/antlr/grammars-v4/tree/master/rust>
10. ocornut/imgui: Dear ImGui. <https://github.com/ocornut/imgui>

附录 A 完整 IR 操作码定义表

表 9: 四元式操作码完整定义

操作码	arg1	arg2	语义
FUNC_BEGIN	函数名	-	函数开始标记
FUNC_END	函数名	-	函数结束标记
LABEL	标签名	-	定义跳转标签
JUMP	目标标签	-	无条件跳转
JZ	条件变量	目标标签	若条件为假则跳转
JNZ	条件变量	目标标签	若条件为真则跳转
ASSIGN	源值	-	赋值: $result = arg1$
ADD	左操作数	右操作数	加法: $result = arg1 + arg2$
SUB	左操作数	右操作数	减法: $result = arg1 - arg2$
MUL	左操作数	右操作数	乘法: $result = arg1 * arg2$
DIV	左操作数	右操作数	除法: $result = arg1 / arg2$
NEG	操作数	-	取负: $result = -arg1$
EQ	左操作数	右操作数	相等: $result = (arg1 == arg2)$
NE	左操作数	右操作数	不等: $result = (arg1 != arg2)$
LT	左操作数	右操作数	小于: $result = (arg1 < arg2)$
LE	左操作数	右操作数	小于等于: $result = (arg1 \leq arg2)$
GT	左操作数	右操作数	大于: $result = (arg1 > arg2)$
GE	左操作数	右操作数	大于等于: $result = (arg1 \geq arg2)$
PARAM	实参值	-	传递实参
CALL	函数名	参数个数	函数调用
RETURN	返回值	-	函数返回
INDEX_LOAD	数组名	索引	读取数组元素: $result = arg1[arg2]$
INDEX_STORE	值	索引	写入数组元素: $arg1[arg2] = arg3$
REF	变量名	-	取引用: $result = \&arg1$
DEREF	引用名	-	解引用: $result = *arg1$
ARRAY_LIT	元素值	位置	数组字面量元素填充
TUPLE_GET	元组名	位置	元组元素访问
BREAK	-	-	跳出循环
CONTINUE	-	-	继续下一次循环
NOP	-	-	空操作

附录 B 完整测试用例索引

表 10: 测试用例与覆盖规则对应表

测试文件	测试类型	覆盖规则	预期结果
test_1_1.rs	语法 + 语义	1.1	通过
test_1_2.rs	语法 + 语义	1.2	通过
test_1_3.rs	语法 + 语义	1.3	通过
test_1_4.rs	语法 + 语义	1.4	通过
test_1_5.rs	语法 + 语义	1.5	通过
test_2.rs	语法 + 语义	2.0–2.3	通过
test_3.rs	语法 + 语义	3.1–3.5	通过
test_4.rs	语法 + 语义	4.1–4.3	通过
test_5.rs	语法 + 语义	5.0–5.4	通过
test_6.rs	语法 + 语义	6.1–6.4	通过
test_8.rs	语法 + 语义	8.1–8.3	通过
test_9.rs	语法 + 语义	9.1–9.3	通过
test_full.rs	语义分析综合	全规则	通过（无错误）
test_errors.rs	语义错误检测	18 类错误	检测到全部错误

运行命令示例

```

1 # 语义分析 + IR生成
2 ./parser --input test_full.rs \
3     --semantic-output semantic.txt \
4     --ir-output ir.txt
5
6 # 仅语法分析（向后兼容）
7 ./parser --input input.rs \
8     --lexer-output lex.tsv \
9     --parser-output tree.txt
10
11 # 静默模式
12 ./parser --input input.rs -q \
13     --semantic-output semantic.txt \
14     --ir-output ir.txt

```

Listing 33: 完整流水线运行命令